A PROJECT REPORT ENTITLED

"CAFE SIMULATOR"

Submitted on the partial fulfillment of requirements for the 6th semester of

BACHELOR OF COMPUTER APPLICATION

Bengaluru North University

BABITHA M [Reg No: U19PL23S0002]

UNDER THE GUIDANCE OF

Prof. NANDHINI M

Assistance Professor Department of Computer Science

CHRIS CANADIAN DEGREE COLLEGE

4th Main Off Ring Road, Kalyan Nagar, Bengaluru-560043

2023-2026

CERTIFICATE

Bengaluru North University

In partial fulfillment of the requirement for the degree of

BACHELOR OF COMPUTER APPLICATION

This is to certify that the work entitled **"Cafe Simulator" c**arried out by **BABITHA M [U19PL23S0002]** student at **CHRIS CANADIAN DEGREE COLLEGE** in partial fulfillment for the Award of Bachelor's in Computer Application of the project submission for VI Semester BCA, Bengaluru North University

Signature of Internal

Signature of HOD

Signature of External

Signature of Principal

STUDENT DECLARATION

I am, **BABITHA M [Reg No. U19PL23S0002]** of VI semester BCA hereby declare that the project **"Cafe Simulator"** is carried out and completed successfully by me during the academic year 2023-2026 under the supervision of Professor **Nandhini M,** Department of Computer Application, **Chris Canadian Degree College.**

PLACE: Bengaluru

DATE: **[Babitha M]**

ACKNOWLEDGEMENT

I am, grateful to our beloved Prof. Rekha Murgan, Principal of **Chris Canadian Degree College,** Kalyan Nagar, whose support and permission were instrumental in undertaking the dissertation. Her guidance throughout this challenging endeavor has been valuable.

I would like to express my sincere gratitude to our respected Prof. Nandhini M, Head of Department for enabling us to embark on this dissertation journey and guiding us through the completion.

I in-debt to the members of the Computer Science department for their guidance and support throughout this endeavor.

CONTENTS

1. ABSTRACT.....07

2. CHAPTER 1: INTRODUCTION..... 08

1.1 : Introduction to the System.....09

1.2 : Problem Statement.....10

1.3 : Objectives of the Project.....11

1.4 : Scope of the Project.....12

3. CHAPTER 2: LITERATURE SURVEY.....14

2.1 : Existing System..... 15

2.2 : Limitations of Existing System.....17

2.3 : Proposed System.....18

4. CHAPTER 3: SYSTEM ANALYSIS.....20

3.1 : Functional Requirements..... 21

3.2 : Non-Functional Requirements.....24

3.3 : Feasibility Study..... 26

5. CHAPTER 4: SYSTEM DESIGN..... 28

4.1 : System Architecture.....29

4.2 : Data Flow Diagram (DFD).....31

4.3 : Entity Relationship Diagram (ER Diagram).....32

4.4 : Class Diagram..... 34

4.5 : Sequence Diagram.....35

6. CHAPTER 5: IMPLEMENTATION..... 36

Frontend Files:

5.1 : home.html page.....38

5.2 : login.html page..... 45

5.3 : menu.html page.....52

5.4 : menu1.html page.....60

5.5 : cart.html page..... 66

5.6 : payment.html page.....74

5.7 : admin.html page.....81

5.8 : style.css.....….87

5.9 : script.js.....89

Backend Files:

5.10.1 : server.js..... 105

5.10.2 : order.js (models)..... 107

5.10.3 : order.js (routes)..... 109

7. CHAPTER 6: TESTING..... 122

6.1: Test Cases..... 123

6.2: Results.....124

8. CHAPTER 7: RESULTS AND DISCUSSION..... 125

9. CHAPTER 8: FUTURE ENHANCEMENT..... 128

10. CONCLUSION..... 131

11. REFERENCES3.....133

ABSTRACT

The **"Cafe Simulator"** is a web-based application developed to provide an efficient and user-friendly online coffee ordering system. The project is designed to simplify the process of ordering food and beverages in a cafe environment through a digital platform. Customers can browse different menu items, add products to a cart, manage item quantities, and place orders using various payment options.

The application is developed using **HTML, CSS, and JavaScrip**t for the frontend to create an attractive and interactive user interface. **Node.js** is used as the backend technology for handling server-side operations, while **MongoDB** is used as the database for storing customer orders and related information.

The system also includes an admin module that enables administrators to monitor customer orders, search order details, delete records, and view complete order information including item names, quantities, prices, and total amount. The project focuses on providing smooth navigation, responsive design, efficient order management, and real-time interaction between frontend and backend systems.

The main objective of this project is to demonstrate the practical implementation of full-stack web development technologies in developing an online café management system. This project helps in reducing manual work, improving order accuracy, and enhancing customer experience through a modern digital solution.

CHAPTER – 01

INTRODUCTION

1.1 INTRODUCTION TO THE SYSTEM

The rapid growth of internet technologies and web applications has transformed the way businesses operate in modern society. The food and beverage industry has also adopted digital solutions to improve customer experience and simplify business operations. Online ordering systems have become an essential part of restaurants and cafés, allowing customers to browse menus, place orders, and complete payments conveniently through web applications.

The "Cafe Simulator" project is a web-based café management and ordering system developed to simulate the functionality of an online coffee shop. The system enables users to explore different coffee and beverage items, add products to a shopping cart, manage quantities, and place orders using a simple and interactive interface. The application is designed to provide an efficient ordering experience for customers while reducing manual work in cafés.

The project is developed using modern web technologies such as HTML, CSS, and JavaScript for frontend design and interactivity. Node.js is used as the backend platform to handle server-side operations, and MongoDB is used as the database for storing order details and customer information.

The application also contains an admin panel where administrators can monitor customer orders, search for specific records, delete unwanted entries, and view complete order details including item names, quantity, price, and total amount. The admin module helps in effective order management and improves system efficiency.

The Cafe Simulator project demonstrates practical implementation of full-stack web development concepts including frontend development, backend integration, database connectivity, and dynamic data handling. The system is designed with a user-friendly interface to ensure smooth navigation and better customer interaction.

This project serves as a model for real-world café ordering systems and highlights the importance of digital solutions in improving customer service and business management in the food industry.

1.2 PROBLEM STATEMENT

Traditional cafe ordering systems mainly depend on manual interaction between customers and staff members. This process can lead to several issues such as long waiting times, incorrect order handling, lack of proper order tracking, and inefficient management of customer data. Manual systems are time-consuming and may reduce the overall customer experience during busy hours.

Many small cafes and restaurants also face difficulties in maintaining order records and monitoring customer transactions efficiently. Customers may find it inconvenient to place orders physically, especially when there is overcrowding or limited staff availability.

To overcome these limitations, there is a need for an efficient web-based ordering system that can automate the ordering process, maintain records digitally, and improve communication between customers and administrators.

The Cafe Simulator project is developed to address these problems by providing an online platform where users can easily browse menu items, place orders, manage carts, and complete payments. The system also provides an admin dashboard for monitoring and managing orders effectively.

1.3 OBJECTIVES OF THE PROJECT

The main objective of the Cafe Simulator project is to develop a user-friendly and efficient online café ordering system using modern web technologies. The project aims to simplify the food ordering process and improve customer satisfaction through digital interaction.

The objectives of the project are as follows:

1. To design and develop a responsive web-based café ordering application.
2. To allow users to browse different menu items and add products to the shopping cart.
3. To provide quantity management and automatic total price calculation.
4. To develop a payment interface for completing orders.
5. To store customer orders securely using MongoDB database.
6. To create an admin panel for viewing and managing customer orders.
7. To implement search and delete functionalities for order management.
8. To provide a simple, attractive, and user-friendly interface for better user experience.
9. To demonstrate practical implementation of frontend and backend web technologies.
10. To reduce manual work and improve order accuracy in café management systems.

1.4 SCOPE OF THE PROJECT

The scope of the Cafe Simulator project is focused on developing a web-based platform for managing café orders digitally. The system is designed to provide an efficient ordering experience for customers and a simple management interface for administrators.

The application allows users to:

- View café menu items
- Add products to cart
- Manage item quantities
- Place orders through the payment page

The admin module enables administrators to:

- Monitor customer orders
- View order details
- Search order records
- Delete unwanted orders

The project mainly focuses on demonstrating the implementation of full-stack web development technologies including frontend design, backend server operations, and database connectivity.

The current version of the project is developed for educational purposes and can be further enhanced in the future with additional features such as online payment gateway integration, mobile application support, user authentication, and live order tracking systems.

The system can be applied in cafes, restaurants, and small food businesses to improve order management and customer interaction through digital platforms.

CHAPTER – 02

LITERATURE SURVEY

2.1 EXISTING SYSTEM

In recent years, online food ordering systems have become highly popular due to the growth of internet technologies and digital platforms. Many restaurants and cafes use web-based and mobile applications to simplify customer ordering processes and improve business efficiency. Existing systems such as Swiggy, Zomato, and other food delivery platforms allow customers to browse menu items, place orders, make payments, and track deliveries online.

Traditional cafe management systems mainly depend on manual operations where customers place orders directly at the counter. These systems require more human interaction and may lead to delays, billing errors, and inefficient order management during busy hours. To overcome these issues, many businesses have adopted computerized ordering systems

Existing online ordering systems generally include the following features:

- Digital menu display

- Shopping cart management

- Online payment options

- Order confirmation

- Customer account management

- Admin order monitoring

These systems improve convenience for customers and reduce manual workload for cafe staff. However, some small-scale businesses may find advanced commercial systems expensive and difficult to customize according to their requirements.

The Cafe Simulator project is inspired by modern online food ordering systems and aims to provide a simplified web-based cafe ordering platform using full-stack web technologies.

2.2 LMITATIONS OF EXISTING SYSTEM

Although existing online food ordering systems provide many advantages, they also have certain limitations and challenges. Traditional manual ordering systems are time-consuming and highly dependent on staff availability. Customers may experience long waiting times, especially during peak business hours.

Some major limitations of existing systems are:

1. Manual order handling may lead to incorrect order placement and billing errors.
2. Lack of proper digital order tracking in small cafés and restaurants.
3. Existing commercial applications may require high maintenance costs.
4. Some systems are complex and difficult for small business owners to manage.
5. Limited customization options in third-party food delivery applications.
6. Dependency on external delivery platforms increases operational costs.
7. Poor user interface design in certain systems may reduce customer experience.
8. Lack of proper admin monitoring features in basic ordering systems.
9. Data management becomes difficult when records are maintained manually.
10. Existing systems may not provide detailed order analysis for administrators.

These limitations create the need for a simple, user-friendly, and efficient café ordering system that can improve customer interaction and simplify order management.

2.3 PROPOSED SYSTEM

The proposed system, "Cafe Simulator," is a web-based cafe ordering application designed to overcome the limitations of traditional ordering methods and existing systems. The system provides a simple and interactive platform where users can browse cafe menu items, add products to a shopping cart, manage quantities, and place orders efficiently.

The proposed system is developed using HTML, CSS, and JavaScript for frontend development, Node.js for backend operations, and MongoDB for storing order data. The application focuses on providing smooth navigation, attractive user interface design, and efficient order management.

Main features of the proposed system include:

- User-friendly home page and login interface
- Digital café menu display
- Add to cart functionality
- Quantity management
- Automatic total price calculation
- Payment processing page
- Admin dashboard for order monitoring
- Search and delete functionalities
- Detailed order information display

The admin module enables administrators to monitor customer orders, search records, delete unnecessary entries, and view complete order details including item names, quantity, price, and total amount.

The proposed system reduces manual work, improves order accuracy, and enhances customer experience through digital automation. It also demonstrates practical implementation of full-stack web development concepts and database connectivity.

The Cafe Simulator project can be further expanded in the future by integrating online payment gateways, mobile application support, customer authentication systems, and live order tracking features.

CHAPTER – 03

SYSTEM ANALYSIS

3.1 FUNCTIONAL REQUIREMENTS

Functional requirements describe the major functions and operations performed by the Cafe Simulator system. These requirements define how the system behaves and how users interact with the application.

The functional requirements of the system are as follows:

1. User Registration and Login

The system provides a login interface where users can enter their credentials to access the application. This helps in controlling user interaction with the system.

2. Menu Display

The application displays a variety of coffee and beverage items with images and prices. Users can browse menu items easily through the menu pages.

3. Add to Cart

Users can add selected items to the shopping cart. The cart stores item details such as product name, quantity, and price.

4. Quantity Management

The system allows users to increase or decrease item quantities directly from the cart page. The total amount is updated automatically.

5. Remove Items

Users can remove unwanted items from the cart before placing the order.

6. Payment System

The application provides a payment page where users can enter details such as name, phone number, address, and payment method before confirming the order.

7. Order Storage

After successful payment, order details are stored in the MongoDB database for future reference.

8. Admin Dashboard

The admin panel allows administrators to:

- View customer orders
- Search orders
- Delete records
- Monitor total revenue
- View complete order details

9. Search Functionality

The system includes a search feature that allows administrators to search customer orders using customer name or phone number.

10. Order Details Display

Administrators can click on customer names to view detailed information including item names, quantity, individual price, and total amount

3.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality attributes and performance characteristics of the Cafe Simulator system. These requirements focus on usability, reliability, performance, and security.

The non-functional requirements of the system are as follows:

1. Performance

The system should respond quickly to user actions such as adding items to the cart, updating quantities, and placing orders.

2 . Usability

The application should provide a simple, attractive, and user-friendly interface for easy navigation and better user experience.

3. Reliability

The system should store customer orders accurately without data loss or duplication.

4. Scalability

The application should support future enhancements such as online payment integration, mobile support, and additional menu items.

5 . Availability

The system should remain accessible whenever users want to place orders or administrators want to monitor records.

6 . Maintainability

The source code should be easy to understand, modify, and maintain for future improvements.

7. Security

Customer order data should be stored securely in the MongoDB database.

8. Compatibility

The application should work properly on different web browsers such as Google Chrome and Microsoft Edge.

9. Responsiveness

The system should adjust properly on different screen sizes including laptops and mobile devices.

10. Data Integrity

The application should maintain accurate and consistent order information throughout the ordering process.

3.3 FEASIBILITY STUDY

A feasibility study is conducted to determine whether the proposed system is practical, beneficial, and economically viable for development and implementation. The feasibility analysis of the Cafe Simulator project includes technical feasibility, economic feasibility, and operational feasibility.

1. Technical Feasibility

The Cafe Simulator project is technically feasible because it is developed using widely available technologies such as HTML, CSS, JavaScript, Node.js, and MongoDB. These technologies are easy to implement and suitable for developing web-based applications.

The required software tools and development platforms are easily accessible and do not require expensive infrastructure. The project can run efficiently on standard computer systems with internet connectivity.

2. Economic Feasibility

The project is economically feasible because it requires minimal development cost. Most of the software tools used in the project are open-source and freely available. The application reduces manual work and improves order management efficiency, which can help businesses save operational costs.

3. Operational Feasibility

The system is operationally feasible because it provides a simple and user-friendly interface for both customers and administrators. Users can easily place orders and manage carts, while administrators can efficiently monitor customer orders through the admin dashboard.

The system is designed to improve customer interaction and simplify cafe management processes. Therefore, it can be successfully implemented in small cafes and food businesses.

4. Schedule Feasibility

The project can be completed within the planned development schedule because the technologies used are simple and suitable for academic projects. Proper planning and modular development approach help in completing the project efficiently within the available time period.

CHAPTER – 04

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

System architecture refers to the overall structure and organization of the Cafe Simulator application. It explains how different components of the system interact with each other to perform various operations efficiently.

The Cafe Simulator system follows a three-layer architecture consisting of:

1. Frontend Layer
2. Backend Layer
3. Database Layer

Frontend Layer

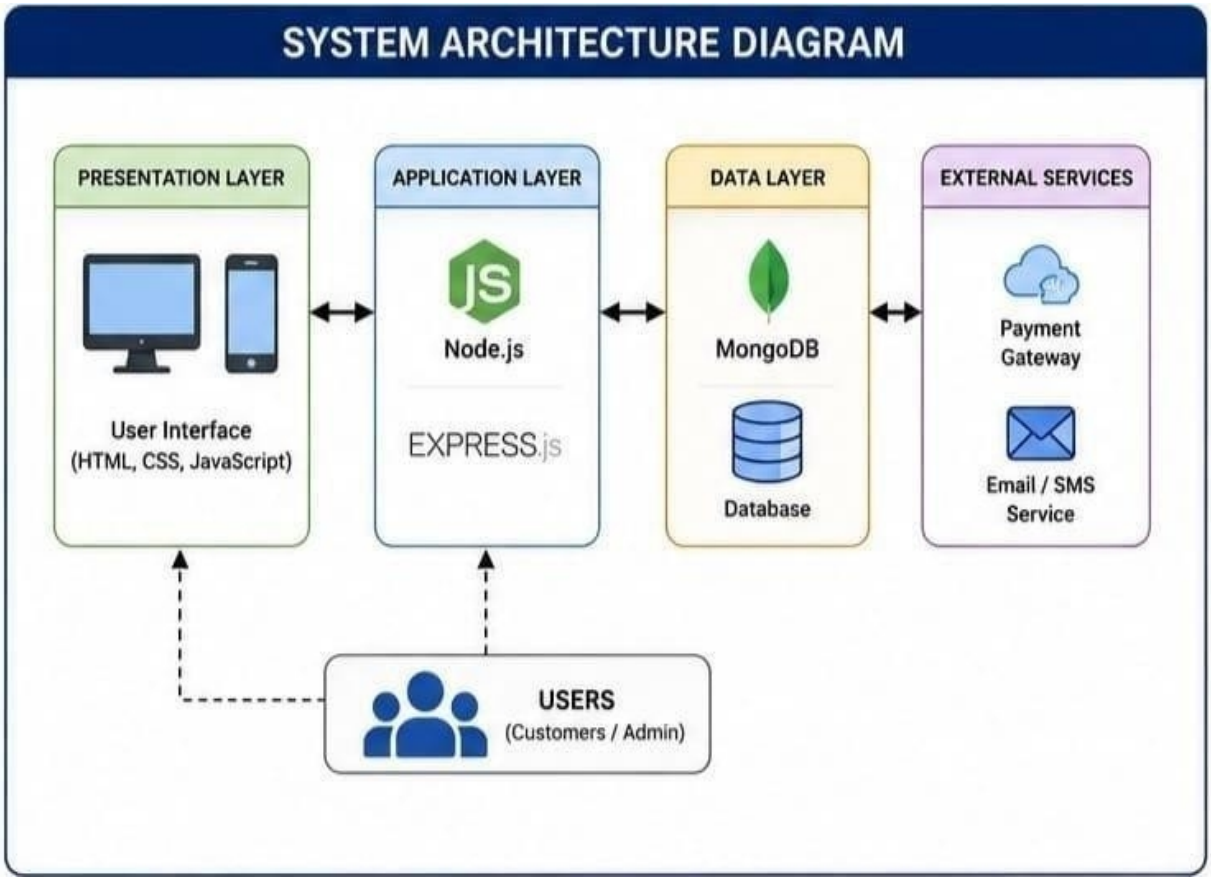
The frontend layer is responsible for providing the user interface of the application. It is developed using HTML, CSS, and JavaScript. This layer allows users to interact with the system by browsing menu items, adding products to the cart, managing quantities, and placing orders.

Backend Layer

The backend layer is developed using Node.js. It handles server-side operations such as processing user requests, storing order details, and communicating with the MongoDB database.

MongoDB is used as the database layer for storing customer order information. It maintains records such as customer details, ordered items, quantities, payment methods, and total amounts.

The system architecture ensures smooth communication between frontend, backend, and database components, providing efficient order processing and data management.

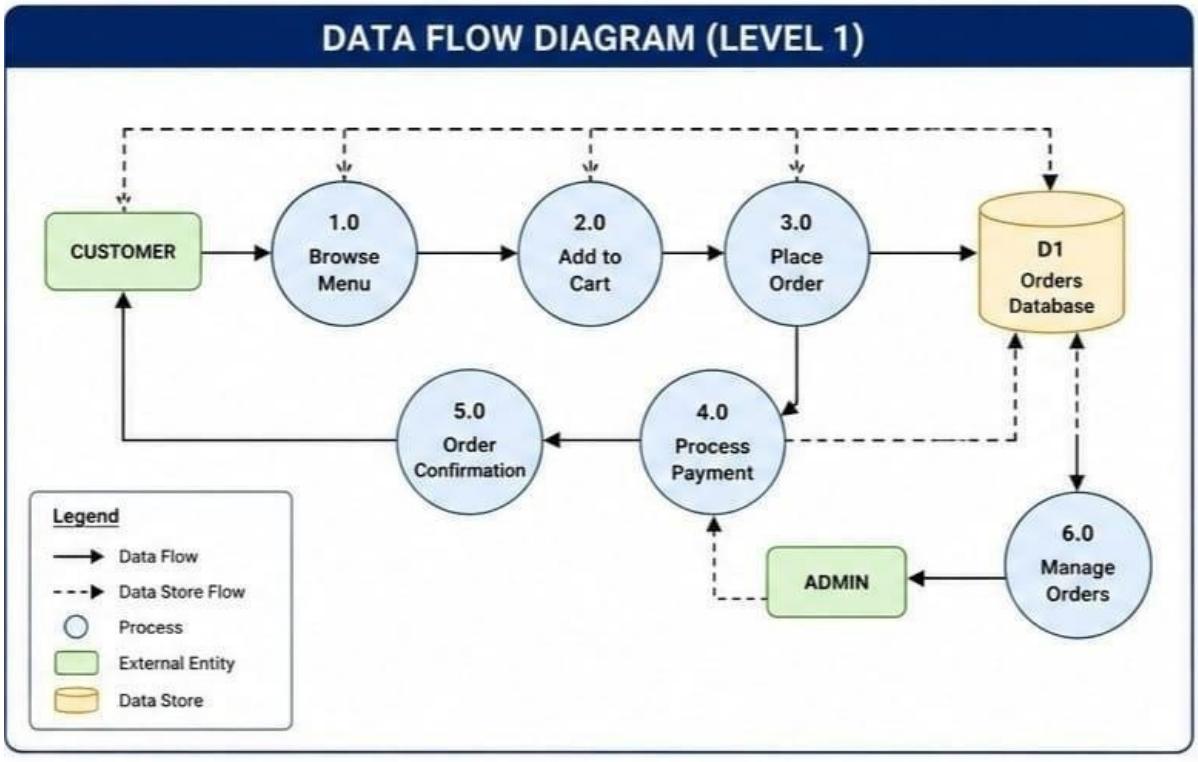


4.2 DATA FLOW DIAGRAM (DFD)

A Data Flow Diagram (DFD) represents the flow of information within the Cafe Simulator system. It shows how data moves between users, processes, and the database.

The DFD illustrates the interaction between customers, the café ordering system, and the admin module. Customers provide inputs such as item selection, quantity management, and payment details. The system processes this information and stores the order details in the MongoDB database.

The admin module retrieves stored order data for monitoring and management purposes.



4.3 ENTITY RELATIONSHIP DIAGRAM (ER DIAGRAM)

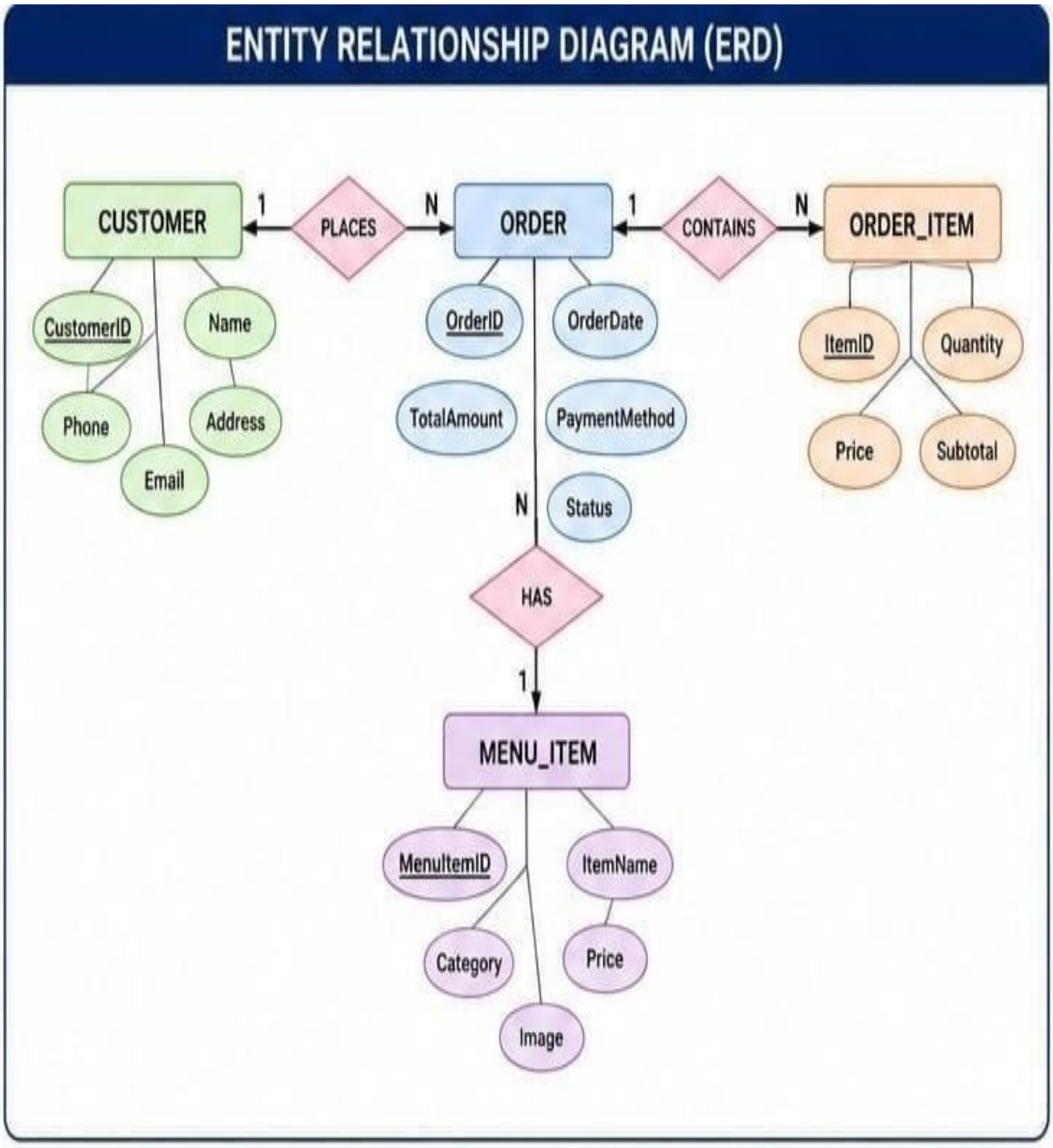
The Entity Relationship Diagram (ER Diagram) represents the relationship between entities present in the Cafe Simulator system. It helps in understanding the database structure and how data is connected within the application.

The major entities in the system are:

- User
- Order
- Items
- Admin

A user can place multiple orders, and each order can contain multiple items. The admin manages and monitors customer orders through the admin panel.

The ER diagram simplifies database design and ensures proper data organization.

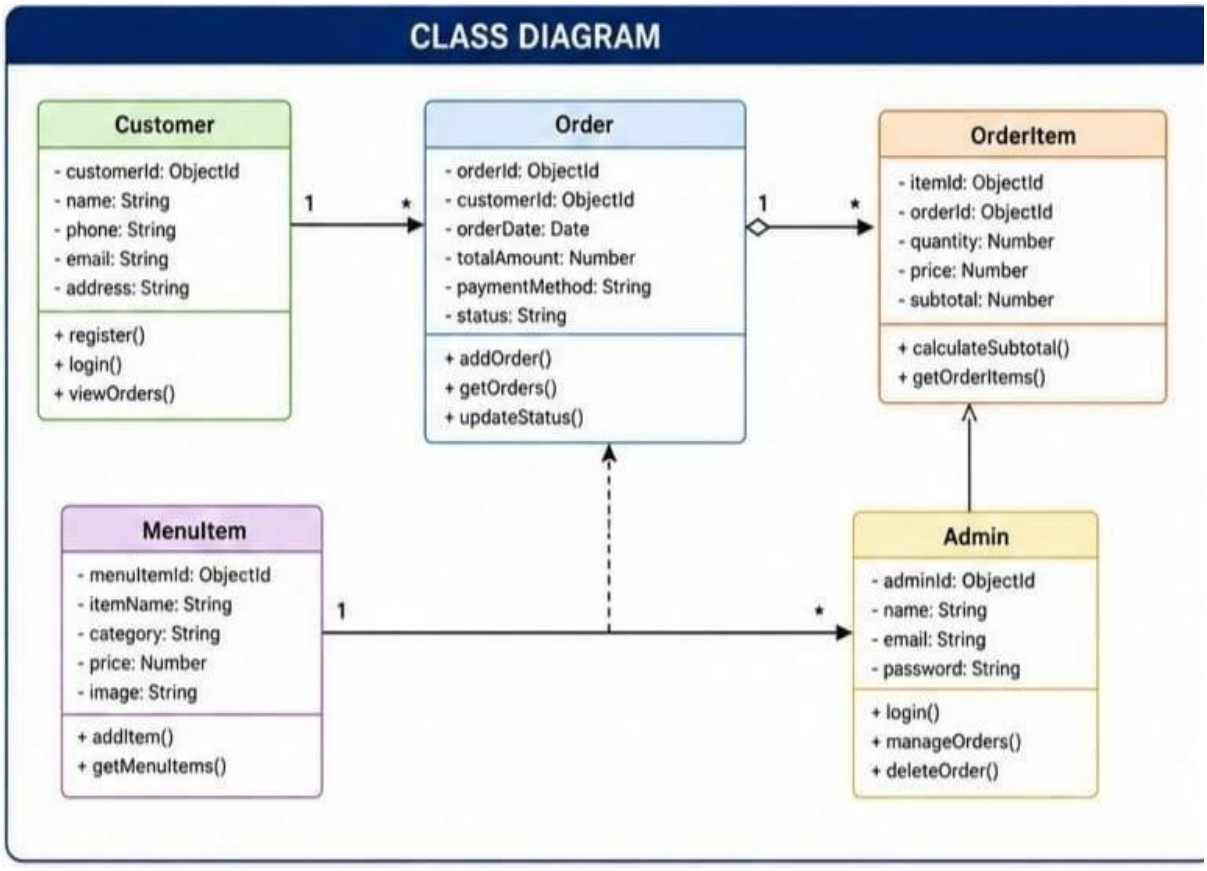

ENTITY RELATIONSHIP DIAGRAM (ERD)
CUSTOMER
1
PLACES
N
ORDER
1
CONTAINS
N
ORDER_ITEM
CustomerID
Name
Phone
Address
Email
OrderID
OrderDate
TotalAmount
PaymentMethod
N
Status
HAS
1
MENU_ITEM
ItemID
Quantity
Price
Subtotal
MenuItemID
ItemName
Category
Price
Image

4.4 CLASS DIAGRAM

The Class Diagram represents the structure of the Cafe Simulator system using classes, attributes, and methods. It helps in understanding object-oriented representation of the system.

The main classes include User, Menu Item, Cart, Order, and Admin. Each class performs specific operations related to the café ordering process.

The class diagram shows how users interact with menu items and cart systems before converting them into confirmed orders managed by the admin module.

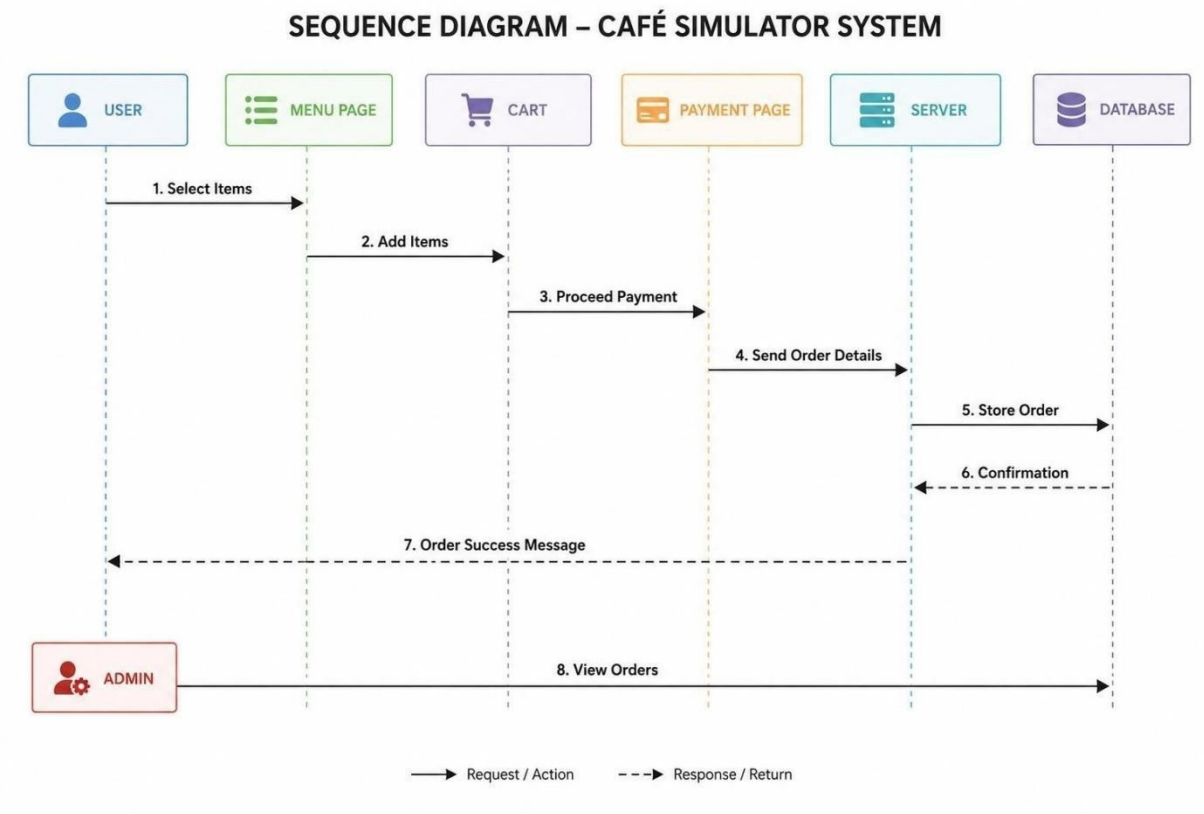


4.5 SEQUENCE DIAGRAM

The Sequence Diagram represents the interaction between different components of the Cafe Simulator system in a sequential manner. It shows how users interact with the application while placing an order.

The sequence begins when the user selects menu items and adds them to the cart. The system processes the order details, stores information in the database, and confirms the order. The admin can later view and manage the stored order details.

The sequence diagram helps in understanding the workflow and communication between system components.



CHAPTER – 05

IMPLEMENTATION

INTRODUCTION

The implementation phase is one of the most important stages in software development. Involves converting system design into a working application using programming languages, development tools, and database technologies. The Cafe Simulator project is implemented as a web-based application that provides an online platform for cafe ordering and management.

The application is developed using HTML, CSS, and JavaScript for frontend development, while Node.js is used for backend server operations. MongoDB is used as the database for storing customer order details and related information.

The implementation of the system focuses on providing an attractive user interface, smooth navigation, efficient order processing, and proper database management. The application contains several modules including Home Page, Login Page, Menu Page, Cart Page, Payment Page, Success Page, and Admin Dashboard.

Each module is designed to perform specific operations within the system. The implementation process also includes integration of frontend and backend technologies to ensure proper communication between users, server, and database.

-FRONTEND FILES:

5.1 home.html page

The Home Page is the main interface of the Cafe Simulator application. It acts as the entry point for users and provides navigation to other pages such as Login, Menu, and Cart pages. The home page is designed with an attractive cafe-themed background image to create a realistic coffee shop experience for users.

The page is developed using HTML and CSS. Various styling features such as hover effects, responsive layouts, custom buttons, and attractive fonts are implemented to improve the user interface and user experience.

The home page provides easy navigation and helps users understand the purpose of the application. Buttons such as “Order Now” and “Explore Menu” guide customers to proceed with ordering food and beverages.

The responsive design ensures that the page works properly on different screen sizes and devices. CSS properties such as background-size, background-position, and background-attachment are used to maintain proper background display even when scrolling.

Home.html code:

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Cafe Simulator</title>


<link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;700&display=swap
" rel="stylesheet">


<style>


body {

   margin: 0;

   font-family: 'Poppins', sans-serif;

   background-image:
url('https://static.vecteezy.com/system/resources/previews/024/529/941/original/coffee-
and-croissant-illustration-ai-generative-free-photo.jpg');

   background-size: cover;

   background-position: center;

   background-repeat: no-repeat;
```

```
    height: 100vh;
    position: relative;
}

body::before
    { content: "";
    position: absolute;
    inset: 0;
    background: rgba(0,0,0,0.2);
}
body * {
    position: relative;
    z-index: 1;
}

.top-right {
    position: absolute;
    top: 20px;
    right: 30px;
    text-align: right;
    font-size: 16px;
    color: white;
}
```

```
.center-content
   { position: absolute;
   top: 45%;
   left: 50%;
   transform: translate(-50%, -50%);
   text-align: center;
}

.center-content h1
   { font-size: 70px;
   font-weight: 700;
   margin: 0;
   color: white;
   white-space: nowrap;
}
.center-content p
   { font-size:
   24px;
   margin: 15px 0 25px;
   color: white;
}
.btn {
   padding: 16px 35px;
   background: linear-gradient(45deg, #6d4c41, #8d6e63);
```

```
    color: white;
    border-radius: 30px;
    font-size: 18px;
    text-decoration: none;
    border: none;
    transition: all 0.3s ease;
}

.btn:hover {
    background: linear-gradient(45deg, #4e342e, #6d4c41);
    transform: translateY(-3px) scale(1.05);
    box-shadow: 0 8px 20px rgba(0,0,0,0.35);
}

.btn:active {
    transform: scale(0.96);
}
.offers {
    position: absolute;
    top: 75%;
    width: 100%;
    text-align: center;
    font-size: 16px;
    color: white;
```

```
}

.bottom {
    position: absolute;
    bottom: 50px;
    width: 100%;
    display: flex;
    justify-content: space-between;
    padding: 0 30px;
    font-size: 15px;
    color: white;
}

</style>
</head>

<body>

<div class="top-right">
    ! 9663926841 <br>
    cafesimulator@gmail.com
</div>

<div class="center-content">
```

```
    <h1>Cafe    Simulator
    <p>There is always a time for Coffee...</p>
    <a href="login.html" class="btn">Order Now</a>
</div>

<div class="offers">
    Free Delivery  |20% Off on Weekends
</div>

<div class="bottom">
    <div>
        4th Main, Kalyan Nagar, Bengaluru
    </div>
    <div>
        Mon-Fri: 9AM-10PM | Sat-Sun: 10AM-11PM
    </div>
</div>

</body>
</html>
```

5.2 login.html page

The Login Page allows users to enter the system securely before accessing menu pages. The page contains username and password input fields along with login buttons for user authentication.

The login page is developed using HTML forms, CSS styling, and JavaScript validation. JavaScript is used to validate whether the input fields are empty before allowing users to continue to the next page.

The page design includes transparent login boxes, modern styling effects, and coffee-themed backgrounds to maintain consistency throughout the application. User-friendly input fields and button animations improve the overall visual appearance of the system.

The login validation process ensures that users enter valid information before proceeding to the cafe menu system.

login.html code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login - Cafe Simulator</title>
  <link rel="stylesheet" href="style.css">
  <style>
    body {
          margin: 0;
      font-family: Arial, sans-serif;
      height: 100vh;
      display: flex;
      justify-content: center;
      align-items:  center;
      background-image:
url('https://static.vecteezy.com/system/resources/thumbnails/024/529/941/small/coffee-
and-croissant-illustration-ai-generative-free-photo.jpg');
      background-size: cover;
      background-position: center;
      background-repeat: no-repeat;
      position: relative;
      overflow: hidden;
```

```
    }
     body::before
    { content: '';
    position: absolute;
    top: 0; left: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(217, 179, 130, 0.6);
    z-index: 0;
   }

  .login-container
   { position:
   relative; z-index:
   1;
  background: rgba(255,255,255,0.3);
   backdrop-filter: blur(10px);
   -webkit-backdrop-filter: blur(10px);

   padding: 40px;
   border-radius: 12px;
    box-shadow: 0 8px 15px rgba(0,0,0,0.2);
   text-align: center;
   width: 300px;
```

```
}

   .login-container  h2
     { margin-bottom:
     20px; color: #3b2f2f;
   }

   .login-container input
     { width: 90%;
     padding: 12px;
     margin: 10px 0;
     border-radius: 6px;
     border: 1px solid #ccc;
     font-size: 1em;
   }

     .login-container .btn
     { width: 95%;
     padding: 12px;
     margin-top: 15px;
     background-color: #6f4e37;
          color: #fff;
     font-size: 1.1em;
     font-weight: bold;
```

```
border: none; border-
radius: 6px; cursor:
pointer;
        text-decoration: none;
        display: inline-block;
        transition: background-color 0.3s ease;
      }
      .login-container  .btn:hover
        { background-color:
        #4b3621;
      }
      @media screen and (max-width: 400px) {
        .login-container
          { width: 90%;
          padding: 30px;
        }
        .login-container input, .login-container .btn
          { width: 100%;
        }
      }
    </style>
  </head>
  <body>
```

```
<div class="login-container">
  <h2>Login</h2>
  <input type="text" placeholder="Username" required>
  <input type="password" placeholder="Password" required>
  <a href="menu.html" class="btn">Next</a>
</div>

<script>
async function login() {
  let username = document.getElementById("username").value;
  let password = document.getElementById("password").value;

  let res = await fetch("http://localhost:5000/api/login",
    { method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ username, password })
  });

  let data = await res.text();

  alert(data);
```

```
    if (data === "Login successful")
        { window.location.href = "menu.html";
    }
}
</script>


</body>
</html>
```

5.3 menu.html page

The Menu Page displays various coffee and beverage items available in the cafe. Each product includes item images, names, prices, and “Add to Cart” buttons.

The menu page is dynamically generated using JavaScript arrays and DOM manipulation techniques. Product cards are displayed in a responsive grid layout to improve visual presentation.

Users can browse multiple products and add selected items to the cart. CSS hover effects and animations are applied to menu cards for better interaction and professional appearance.

The menu page improves customer convenience by providing a digital catalog of café products with proper categorization and navigation.

menu.html code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Café Menu - Cafe Simulator</title>
  <link rel="stylesheet" href="style.css">

  <style>
    body {
      margin: 0;
      font-family: Arial, sans-serif;
      background-color: #f5e0c3; /* light brown background */
    }

    header {
      background-color: #6f4e37;
      color: white;
      padding: 15px;
      text-align: center;
    }
```

```
header nav a
 { color: white;
 margin: 0 15px;
 text-decoration: none;
 font-weight: bold;
}

header nav a:hover {
 text-decoration: underline;
}

.menu-container
 { display: flex;
 flex-wrap: wrap;
 justify-content: center;
 padding: 20px;
}

.menu-item
 { background-color:
 #fff; border-radius:
 10px;
 box-shadow: 0 5px 15px rgba(0,0,0,0.1);
 margin: 15px;
 text-align: center;
```

```
  width: 180px;
  padding: 10px;
  transition: transform 0.2s;
}

.menu-item:hover
  {transform:
  scale(1.05);
}

.menu-item img
  { width: 100%;
  height: auto;
  object-fit: cover;
  border-radius: 10px;
}

.menu-item p
  { margin: 5px 0;
  font-weight: bold;
  font-size: 1em;
  color: #3b2f2f;
}

.menu-item button {
```

```
    background-color: #6f4e37;
    color: white;
    border: none;
    border-radius: 6px;
    padding: 8px 12px;
    cursor: pointer;
    margin-top: 8px;
    font-weight: bold;
    transition: background-color 0.3s ease;
  }

  .menu-item button:hover
   { background-color:
   #4b3621;
  }

  @media screen and (max-width: 500px) {
   .menu-item
     { width: 90%;
   }
  }
 </style>
</head>
<body>
```

```
<header>
  <div class="cart-icon">
    <a href="cart.html">cart (<span id="cart-count">0</span>)</a>
  </div>

  <h1>Café Menu</h1>
  <nav>
    <a href="home.html">Home</a>
    <a href="menu1.html">More Menu</a>
    <a href="cart.html">Cart</a>
  </nav>
</header>

<main class="menu-container" id="menu-container">
  <!-- Menu items will be inserted here -->
</main>

<script>
  // List of 25 coffee items with their image names and prices
  const menuItems = [
    {name: "affogato", price: 120, image: "images/affogato.jpg"},
    {name: "almond", price: 130, image: "images/almond.jpg"},
    {name: "americano", price: 140, image: "images/americano.jpg"},
    {name: "black", price: 150, image: "images/black.jpg"},
```

```
    {name: "cappuccino", price: 160, image: "images/cappuccino.jpg"},
    {name: "caramel latte", price: 110, image: "images/caramel latte.jpg"},
    {name: "chocolate", price: 115, image: "images/chocolate.jpg"},
    {name: "coconut", price: 125, image: "images/coconut.jpg"},
    {name: "cold brew", price: 135, image: "images/cold brew.jpg"},
    {name: "espresso", price: 145, image: "images/espresso.jpg"},
    {name: "flat white", price: 150, image: "images/flat white.jpg"},
    {name: "frappe", price: 155, image: "images/frappe.jpg"},
    {name: "hazelnut coffee", price: 160, image: "images/hazelnut coffee.jpg"},
    {name: "honey", price: 165, image: "images/honey.jpg"},
    {name: "iced mocha", price: 170, image: "images/iced mocha.jpg"},
    {name: "irish coffee", price: 175, image: "images/irish coffee.jpg"},
    {name: "latte", price: 180, image: "images/latte.jpg"},
    {name: "macchiato", price: 185, image: "images/macchiato.jpg"},
    {name: "milk", price: 190, image: "images/milk.jpg"},
    {name: "mint", price: 195, image: "images/mint.jpg"},
    {name: "mocha", price: 200, image: "images/mocha.jpg"},
    {name: "pumpkin", price: 205, image: "images/pumpkin.jpg"},
    {name: "turkish", price: 210, image: "images/turkish.jpg"},
    {name: "vanilla latte", price: 215, image: "images/vanilla latte.jpg"},
    {name: "iced latte", price: 220, image: "images/iced latte.jpg"},
  ];
  const menuContainer = document.getElementById("menu-container");
```

```
    menuItems.forEach(item => {
      const div = document.createElement("div");
      div.className  = "menu-item";
      div.innerHTML = `
        <img src="${item.image}" alt="${item.name}">
        <p>${item.name}</p>
        <p>₹${item.price}</p>
        <button onclick="addToCart('${item.name}', ${item.price})">Add to
Cart</button>
      `; menuContainer.appendChild(div);
    });

  </script>

  <div class="next-page">
    <a href="menu1.html">Next</a>
  </div>
  <div id="notification"  class="notification"></div>
  <script src="script.js"></script>

</body>
</html>
```

60

5.4 menu1.html code:

```
<!DOCTYPE html>
<html>
<head>
 <title>Snacks Menu</title>
 <link rel="stylesheet" href="style.css">
</head>

<body class="menu1-page">

 <div class="cart-icon">
  <a href="cart.html">Cart (<span id="cart-count">0</span>)</a>
 </div>

 <h1 style="text-align:center;">Snacks & Bakery</h1>
 <div class="menu-container">

  <div class="item"> <img src="images/Coffee Cake.jpg">
   <h3>Coffee Cake</h3>
   <p>₹180</p> <button onclick="addToCart('Coffee Cake',180)">Add to
Cart</button>
  </div>
```

```
<div class="item"> <img src="images/Cheese Danish.jpg">
<h3>Cheese Danish</h3>
<p>₹150</p> <button onclick="addToCart('Cheese Danish',150)">Add to Cart</button>
</div>

<div class="item"> <img src="images/Pumpkin Muffin.jpg">
<h3>Pumpkin Muffin</h3>
<p>₹120</p> <button onclick="addToCart('Pumpkin Muffin',120)">Add to Cart</button>
</div>

<div class="item"> <img src="images/Glazed Doughnut.jpg">
<h3>Glazed Doughnut</h3>
<p>₹100</p> <button onclick="addToCart('Glazed Doughnut',100)">Add to Cart</button>
</div>

<div class="item"> <img src="images/Blueberry Muffin.jpg">
<h3>Blueberry Muffin</h3>
<p>₹130</p> <button onclick="addToCart('Blueberry Muffin',130)">Add to Cart</button>
</div>

<div class="item"> <img src="images/Butter Croissant.jpg">
```

```
    <h3>Butter Croissant</h3>
    <p>₹140</p> <button onclick="addToCart('Butter Croissant',140)">Add to Cart</button>
  </div>

  <div class="item"> <img src="images/Swiss Croissant.jpg">
    <h3>Swiss Croissant</h3>
    <p>₹150</p> <button onclick="addToCart('Swiss Croissant',150)">Add to Cart</button>
  </div>

  <div class="item"> <img src="images/Almond Croissant.jpg">
    <h3>Almond Croissant</h3>
    <p>₹160</p> <button onclick="addToCart('Almond Croissant',160)">Add to Cart</button>
  </div>

  <div class="item"> <img src="images/Chocolate Croissant.jpg">
    <h3>Chocolate Croissant</h3>
    <p>₹170</p> <button onclick="addToCart('Chocolate Croissant',170)">Add to Cart</button>
  </div>

  <div class="item"> <img src="images/Cookie.jpg">
    <h3>Cookie</h3>
```

```
    <p>₹80</p> <button onclick="addToCart('Cookie',80)">Add to Cart</button>
  </div>

  <div class="item"> <img src="images/Choco Chip Cookie.jpg">
    <h3>Choco Chip Cookie</h3>
    <p>₹90</p> <button onclick="addToCart('Choco Chip Cookie',90)">Add to
Cart</button>
  </div>

  <div class="item"> <img src="images/Brownie.jpg">
    <h3>Brownie</h3>
    <p>₹140</p> <button onclick="addToCart('Brownie',140)">Add to Cart</button>
  </div>

  <div class="item"> <img src="images/Lime Coconut Bar.jpg">
    <h3>Lime Coconut Bar</h3>
    <p>₹110</p> <button onclick="addToCart('Lime Coconut Bar',110)">Add to
Cart</button>
  </div>

  <div class="item"> <img src="images/Cinnamon Roll.jpg">
    <h3>Cinnamon Roll</h3>
    <p>₹130</p> <button onclick="addToCart('Cinnamon Roll',130)">Add to
Cart</button>
  </div>
```

```
<div class="item"> <img src="images/Caramel Waffles.jpg">
  <h3>Caramel Waffles</h3>
  <p>₹160</p> <button onclick="addToCart('Caramel Waffles',160)">Add to Cart</button>
</div>

<div class="item"> <img src="images/Grilled  Cheese.jpg">
  <h3>Grilled Cheese</h3>
  <p>₹150</p> <button onclick="addToCart('Grilled Cheese',150)">Add to Cart</button>
</div>

<div class="item"> <img src="images/Spinach Feta Wrap.jpg">
  <h3>Spinach Feta Wrap</h3>
  <p>₹170</p> <button onclick="addToCart('Spinach Feta Wrap',170)">Add to Cart</button>
</div>

<div class="item"> <img src="images/Jalapeno Chicken Pocket.jpg">
  <h3>Jalapeno Chicken Pocket</h3>
  <p>₹180</p> <button onclick="addToCart('Jalapeno Chicken Pocket',180)">Add to Cart</button>
</div>
```

```
    <div class="item"> <img src="images/French Fries.jpg">
      <h3>French Fries</h3>
      <p>₹90</p> <button onclick="addToCart('French Fries',90)">Add to
Cart</button>
    </div>

    <div class="item"> <img src="images/Veggie Wrap.jpg">
      <h3>Veggie Wrap</h3>
      <p>₹140</p> <button onclick="addToCart('Veggie Wrap',140)">Add to
Cart</button>
    </div>
  </div>
  <div class="nav-buttons"> <a href="menu.html"><button>Back</button></a> <a
href="cart.html"><button>Next</button></a>
  </div>
  <div id="notification"  class="notification"></div>
  <script src="script.js"></script>

</body>
</html>
```

5.5 cart.html page

The Cart Page allows users to view selected products before placing orders. Users can increase or decrease product quantity, remove items from the cart, and view the total amount dynamically calculated by the system.

The cart functionality is implemented using JavaScript local storage for temporary data storage. Dynamic quantity management ensures that total price updates automatically whenever users modify item quantities.

The page is designed using centered layouts, transparent cart containers, and coffee-themed background images. Responsive CSS design ensures that the cart page maintains proper alignment and appearance on different devices.

The cart system improves order management and provides better user interaction before payment confirmation.

cart.html code:

```
<!DOCTYPE html>
<html>
<head>
<title>Your Cart</title>
<link rel="stylesheet" href="style.css">

<style>

body {
   margin: 0;
   font-family: 'Poppins', sans-serif;
   background-image:
url('https://static.vecteezy.com/system/resources/previews/024/529/941/original/coffee-
and-croissant-illustration-ai-generative-free-photo.jpg');
   background-size: cover;
   background-position: center;
   background-repeat: no-repeat;
   background-attachment: fixed;

   display: flex;
   flex-direction: column;
   align-items:  center;
```

```
}

h1 {
   text-align: center;
   color: white;
     margin-top: 30px;
}

.cart-container
   { margin-top:
   30px; width:
   380px;
   text-align: center;
}
.cart-container::-webkit-scrollbar
   { width: 6px;
}

.cart-container::-webkit-scrollbar-thumb
   { background: #6d4c41;
   border-radius: 10px;
}

.cart-item {
```

```
    display: grid;
    grid-template-columns: 1fr auto auto auto;
    align-items:  center;

    gap: 10px;
    padding: 10px;
    margin: 10px 0;

    background: rgba(0,0,0,0.4);
    border-radius: 10px;
    color: white;
}

.cart-item button
    { background:
    #8d6e63; border:
    none; padding: 5px
    10px; color: white;
    border-radius: 5px;
    cursor: pointer;
}

.cart-item button:hover
    { background: #6d4c41;
```

```
}

.cart-item span:first-child
   { font-weight: 500;
}


h2 {
   color: white;
   text-align: center;
   margin-top: 15px;
}

.nav-buttons {
   text-align: center;
   margin-top: 20px;
}

.nav-buttons button
   { padding: 10px 18px;
   margin: 5px;
   background: linear-gradient(45deg, #6d4c41, #8d6e63);
   color: white;
   border: none;
```

```
    border-radius: 25px;
    font-size:  14px;
    cursor: pointer;
    transition: 0.3;
}

 .nav-buttons button:hover
    { transform: scale(1.05);
    box-shadow: 0 5px 15px rgba(0,0,0,0.3);
}

.remove-btn
    { background:
    #a1887f; padding:
    6px 12px; border-
    radius: 6px;
}

.remove-btn:hover
    { background:
    #5d4037;
}

#cart-items:empty::before
    { content: "Your cart is empty ;"
```

```
    display: block;
    text-align: center;
    margin-top: 20px;
}

</style>

</head>

<body onload="displayCart()">

<h1>🛒 Your Cart</h1>

<div class="cart-container">

    <div id="cart-items"></div>

    <h2>Total: ₹<span id="total">0</span></h2>

</div>

<div class="nav-buttons">
    <a href="menu1.html"><button>Back</button></a>
    <button onclick="clearCart()">Clear</button>
```

```
    <a href="payment.html"><button>Payment</button></a>
</div>


<script src="script.js"></script>


</body>
</html>
```

5.6 payment.html page

The Payment Page is responsible for collecting customer details and processing orders. Users must enter personal information such as name, phone number, address, and preferred payment method.

Different payment options such as Card Payment, UPI, QR Code, and Cash on Delivery are included in the application. JavaScript validation ensures that users enter all required information before completing payment.

When users select the QR payment option, the system dynamically displays the QR code image using JavaScript. The payment page uses attractive styling and consistent coffee-themed background images to maintain UI design throughout the system.

After successful payment, customer order details are sent to the backend server and stored in the MongoDB database.

payment.html code:

```
<!DOCTYPE html>
<html>
<head>
<title>Payment</title>

<style>

body
 { margin:
 0;
 font-family: Arial, sans-serif;
 height: 100vh;
 display: flex;
 justify-content: center;
 align-items:  center;

 background-image:
url('https://static.vecteezy.com/system/resources/thumbnails/024/529/941/small/coffee-and-croissant-illustration-ai-generative-free-photo.jpg');
 background-size: cover;
 background-position: center;
 background-repeat: no-repeat;
```

```
  position: relative;
}

/* Bright overlay like login page */
body::before {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(217, 179, 130, 0.6);
  z-index: 0;
}

/* PAYMENT BOX */
.payment-container
  { position: relative;
  z-index: 1;

  background: rgba(255, 255, 255, 0.9);
  padding: 30px;
  border-radius: 12px;
  width: 350px;
```

```
  box-shadow: 0 8px 20px rgba(0,0,0,0.3);
  text-align: center;
}

/* INPUTS */
.payment-container input,
.payment-container textarea,
.payment-container select
  { width: 90%;
  padding: 10px;
  margin: 10px 0;

  border-radius: 6px;
  border: 1px solid #ccc;
}

/* BUTTON */
.pay-btn { width:
  100%;
  padding: 12px;
  margin-top: 15px;

  background: linear-gradient(45deg, #6f4e37, #a1887f);
```

```
  color: white;
  border: none;
  border-radius: 6px;

  font-size: 16px;
  font-weight: bold;
  cursor: pointer;

  transition: 0.3s;
}

.pay-btn:hover
  { transform: scale(1.05);
  background: linear-gradient(45deg, #4e342e, #8d6e63);
}

.loading-box
    { display: none;
    margin-top: 15px;
    font-weight: bold;
    color: #6f4e37;
}

/* QR */
```

```
.qr-box {
    margin-top: 10px;
}

.qr-img {
    width: 150px;
    border-radius: 10px;
}

</style>

</head>

<body>

<h1>Payment Details </h1>

<div class="payment-container">

<label>Full Name</label>
<input type="text" id="name" placeholder="Enter your name">

<label>Phone Number</label>
<input type="text" id="phone" placeholder="Enter phone number">
```

```
<label>Address</label>

<textarea id="address" placeholder="Enter delivery address"></textarea>


<select id="paymentMethod" onchange="showPaymentFields()">

  <option value="COD">Cash on Delivery</option>

  <option value="UPI">UPI</option>

  <option value="CARD">Card</option>

  <option value="QR">QR Code</option>

</select>


<div id="paymentDetails"></div>

<button class="pay-btn" onclick="completePayment()">Pay Now</button>


<div id="loading" class="loading-box">

🍳Preparing your order...

</div>


</div>

<script src="script.js"></script>


</body>

</html>
```

5.7 admin.html page

The Admin Page is developed for monitoring and managing customer orders efficiently. Administrators can view all customer orders stored in the MongoDB database through the admin dashboard.

The admin panel displays:

- Customer Name

- Phone Number

- Address

- Payment Method

- Total Amount

- Date and Time

Additional functionalities include:

- Search orders

- Delete orders

- View total revenue

- View detailed item information

Administrators can click customer names to view complete ordered item details including item names, quantity, prices, and total amount.

The admin module improves order tracking and simplifies management operations in the cafe system. The page is designed using tables, styled buttons, and responsive layouts for professional appearance.

admin-login.html code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Admin Login</title>
  <link rel="stylesheet" href="style.css">

  <link href="https://fonts.googleapis.com/css2?family=Poppins&display=swap"
rel="stylesheet">
</head>

<body class="login-page">

<div class="overlay">
  <div class="login-container">

    Cafe Admin</h1>

    <input type="text" id="username" placeholder="Enter username">
    <input type="password" id="password" placeholder="Enter password">

    <button onclick="adminLogin()">Login</button>
```

```
  </div>
</div>

<script src="script.js"></script>

</body>
</html>
```

admin.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>Cafe Admin Panel</title>
  <link rel="stylesheet" href="style.css">

  <!-- Google Font (coffee style) -->
  <link href="https://fonts.googleapis.com/css2?family=Poppins&display=swap"
rel="stylesheet">
</head>

<body class="admin-page">
<h1>☕ Cafe  Orders  Dashboard</h1>

<script>
  if (localStorage.getItem("isAdmin") !== "true")
    { window.location.href = "admin-login.html";
  }
</script>

<div class="header-bar">
```

```
 <input
  type="text"
  id="searchInput"
  p la ce ho ld er=' Search by name or phone..."
  onkeyup="searchOrders()"
 >

 <div class="stats-right">
  <div class="order-icon">
   Orders: <strong id="orderCount">0</strong></span>
  </div>

  <div class="revenue-box">
   x ₹<span id="totalRevenue">0</span>
  </div>

  <button class="logout-btn" onclick="logout()">Logout</button>

 </div>
</div>

<table id="ordersTable">
 <thead>
  <tr>
```

```
      <th>Name</th>
      <th>Phone</th>
      <th>Address</th>
      <th>Payment</th>
      <th>Total</th>
      <th>Date & Time</th>
      <th>Action</th>
    </tr>
  </thead>

  <tbody id="orders"></tbody>
</table>

<script src="script.js"></script>

<script>
document.addEventListener("DOMContentLoaded", function ()
  { loadOrders();
});
</script>

</body>
</html>
```

5.8 style.css

The "style.css" file is used to design and style the entire Cafe Simulator web application. It controls the appearance, layout structure, fonts, colors, spacing, responsiveness, hover effects, animations, and background images used throughout the project.

The stylesheet ensures a consistent coffee-themed user interface across all pages including Home Page, Login Page, Menu Page, Cart Page, Payment Page, Success Page, and Admin Dashboard. Different CSS properties and styling techniques are implemented to improve user interaction and provide a modern and professional appearance.

The CSS file contains styling for:

- Navigation bars
- Buttons
- Input fields
- Tables
- Menu cards
- Cart layouts
- Admin dashboard
- Payment forms
- Background images
- Responsive design

Hover effects and transition animations are used to improve user experience and visual interaction. Flexbox layouts and responsive media queries help maintain proper alignment and page structure on different devices such as desktops, laptops, tablets, and mobile phones.

The style sheet also includes transparent containers, box shadows, rounded corners, and responsive design elements that make the application visually attractive and user-friendly.

Features Implemented Using CSS

- Responsive Web Design
- Coffee-Themed Background Images
- Navigation Bar Styling
- Hover Effects and Animations
- Cart Layout Design
- Admin Dashboard Styling
- Button Styling
- Form Styling
- Payment Page Design
- Mobile Screen Support
- Notification Popup Styling
- QR Code Image Styling

5.9 script.js

The "script.js" file contains the JavaScript logic used in the Cafe Simulator project. It is responsible for implementing dynamic functionalities and interactive features within the application.

JavaScript is used to manage user interactions, cart operations, payment processing, notifications, validations, and admin functionalities. The script file helps establish communication between different pages and ensures smooth execution of frontend operations.

The "script.js" file performs several important tasks including:

- Adding items to the cart
- Removing products from the cart
- Updating item quantity
- Calculating total amount
- Displaying notifications
- Validating payment forms
- Managing localStorage data
- Loading customer orders
- Searching orders in admin panel
- Deleting order records
- Displaying detailed order information

The application uses JavaScript localStorage to temporarily store cart items before payment confirmation. Fetch API is used for sending and retrieving data between frontend pages and backend server APIs.

The script file improves interactivity and enables dynamic updates without reloading the webpage. JavaScript functions are modularly designed to simplify maintenance and future enhancements of the project.

Features Implemented Using JavaScript

- Add to Cart Functionality
- Remove Item Functionality
- Quantity Increment and Decrement
- Total Price Calculation
- Form Validation
- Notification Popup Messages
- Payment Processing
- Fetch API Integration
- Order Data Retrieval
- Search Functionality
- Delete Order Functionality
- Dynamic Cart Display
- QR Code Display Logic

script.js code:

```
document.addEventListener("DOMContentLoaded", function ()
   { updateCartCount();
   displayCart();
});

function addToCart(item, price) {

   let cart = JSON.parse(localStorage.getItem("cart")) || [];

   // check if item already exists
   let existingItem = cart.find(i => i.name === item);

   if (existingItem) {
      existingItem.qty = (existingItem.qty || 1) + 1;
   } else {
      cart.push({ na
         me: item,
         price: price,
         qty: 1
      });
   }
```

```
    localStorage.setItem("cart", JSON.stringify(cart));

    showNotification(item + " added to cart");
    updateCartCount();
}

function displayCart() {

    let cartItems = document.getElementById("cart-items");
    let totalElement = document.getElementById("total");

    if (!cartItems || !totalElement) return;

    let cart = JSON.parse(localStorage.getItem("cart")) || [];

    let total = 0;
    cartItems.innerHTML = "";

    cart.forEach((item, index) => {

        let qty = item.qty || 1;
        let itemTotal = item.price * qty;
```

```
        total += itemTotal;


        cartItems.innerHTML += `
        <div class="cart-item">
            <span>${item.name}</span>


            <div>
                <button  onclick="decreaseQty(${index})">-</button>
                <span> ${qty} </span>
                <button onclick="increaseQty(${index})">+</button>
            </div>


            <span>₹${itemTotal}</span>


            <button class="remove-btn"
onclick="removeItem(${index})">Remove</button>
        </div>
        `;
    });


    totalElement.innerText = total;
}


function increaseQty(index) {
```

```
    let cart = JSON.parse(localStorage.getItem("cart")) || [];
    cart[index].qty = (cart[index].qty || 1) + 1;
    localStorage.setItem("cart", JSON.stringify(cart));
    displayCart();
    updateCartCount();
}

function decreaseQty(index) {
    let cart = JSON.parse(localStorage.getItem("cart")) || [];

    if ((cart[index].qty || 1) > 1)
        { cart[index].qty -= 1;
    } else {
        cart.splice(index, 1);
    }

    localStorage.setItem("cart", JSON.stringify(cart));
    displayCart();
    updateCartCount();
}

function removeItem(index) {
    let cart = JSON.parse(localStorage.getItem("cart")) || [];
    cart.splice(index, 1);
```

```
    localStorage.setItem("cart", JSON.stringify(cart));
    displayCart();
    updateCartCount();
}

function clearCart() {
    let confirmClear = confirm("Clear all items?");
    if (confirmClear) {
        localStorage.removeItem("cart");
        displayCart(); updateCartCount();
    }
}

function updateCartCount() {
    let cart = JSON.parse(localStorage.getItem("cart")) || [];
    let count = cart.reduce((sum, item) => sum + (item.qty || 1), 0);

    let cartCount = document.getElementById("cart-count");
    if (cartCount) {
        cartCount.innerText = count;
    }
}
```

```
function showNotification(message) {
    let notification = document.getElementById("notification");

    if (notification)
        { notification.innerText = message;
        notification.style.display = "block";

        setTimeout(() =>
            { notification.style.display = "none";
        }, 2000);
    }
}

async function completePayment() {

    let cart = JSON.parse(localStorage.getItem("cart")) || [];
    let name = document.getElementById("name").value;
    let phone = document.getElementById("phone").value;
    let address = document.getElementById("address").value;

    if (!name || !phone || !address)
        { alert("Please fill all fields");
        return;
```

```
    }
    const orderData =
        { items: cart,
        total: cart.reduce((sum, item) => sum + (item.price * (item.qty || 1)), 0),
        name: name,
        phone: phone,
        address: address,
        paymentMethod: document.getElementById("paymentMethod").value
    };

    fetch("http://localhost:5000/api/order", { method:
        "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify(orderData)
    }).catch(err => console.error(err));


    localStorage.removeItem("cart");
    window.location.href = "success.html";
}

function showPaymentFields() {
```

```
const method = document.getElementById("paymentMethod").value;
const container = document.getElementById("paymentDetails");

if (method === "CARD")
  { container.innerHTML = `
    <input type="text" placeholder="Card Number"><br>
    <input type="text" placeholder="Expiry Date"><br>
    <input type="text" placeholder="CVV"><br>
  `;
}
else if (method === "UPI")
  { container.innerHTML =
  `
    <input type="text" placeholder="Enter UPI ID"><br>
  `;
}
else if (method === "QR")
  { container.innerHTML = `
    <div class="qr-box">
      <p>Scan & Pay</p>
      <img src="images/QR.jpg" class="qr-img">
      <p style="font-size:12px;">Use any UPI app</p>
    </div>
  `;
```

```
    }
    else {
        container.innerHTML = "";
    }
}

function adminLogin() {
  const username = document.getElementById("username").value;
  const password = document.getElementById("password").value;

  if (username === "admin" && password === "1234")
    { localStorage.setItem("isAdmin", "true");
    window.location.href = "admin.html";
  } else {
    alert("Invalid credentials!");
  }
}

async function loadOrders()
  { try {
    const res = await fetch("http://localhost:5000/api/orders");
    const orders = await res.json();

    console.log("Orders:", orders);
```

```
const ordersTable = document.getElementById("orders");
if (!ordersTable) return;

ordersTable.innerHTML = "";

let total = 0;

orders.forEach(order =>
 { total += order.total;

  const row = `
   <tr>
    <td onclick='showItems(${JSON.stringify(order.items)})'
      style="cursor:pointer; color:#6f4e37; font-weight:bold; text-decoration:
underline;">
      ${order.name}
    </td>
    <td>${order.phone}</td>
    <td>${order.address}</td>
    <td>${order.paymentMethod}</td>
    <td>₹${order.total}</td>
    <td>${new Date(order.date).toLocaleString()}</td>
    <td>
```

```
          <button class="delete-btn"
onclick="deleteOrder('${order._id}')">Delete</button>
        </td>
       </tr>
     `;

     ordersTable.innerHTML += row;
   });

   document.getElementById("orderCount").innerText = orders.length;
   document.getElementById("totalRevenue").innerText  = total;

  } catch (error) {
   console.error("Error loading orders:", error);
  }
}

async function deleteOrder(id)
 { try {
   await fetch(`http://localhost:5000/api/order/${id}`,
    { method: "DELETE"
   });

   alert("Order deleted!");
```

```
    loadOrders(); // refresh table

  } catch (error) {
    console.error("Delete error:", error);
  }
}

function searchOrders() {
  const input = document.getElementById("searchInput").value.toLowerCase();
  const rows = document.querySelectorAll("#orders tr");

  rows.forEach(row => {
    const name = row.children[0].innerText.toLowerCase();
    const phone = row.children[1].innerText.toLowerCase();

    if (name.includes(input) || phone.includes(input))
      { row.style.display = "";
    } else {
      row.style.display = "none";
    }
  });
}
```

```
function showItems(items) {

  let message = "🧾 Order Details:\n\n";

  let total = 0;

  items.forEach(item =>

    { let qty = item.qty || 1;

    let itemTotal = item.price * qty;


    total += itemTotal;


   message += `• ${item.name} x ${qty} = ₹${itemTotal}\n`;

  });


  message += "\n---------- \n";

  message += `Total Amount: ₹${total}`;


  alert(message);

}
```

BACKEND FILES:

Backend files play an important role in the Cafe Simulator project by handling server-side operations, database communication, API management, and order processing functionalities.

The backend of the project is developed using Node.js and Express.js framework. MongoDB is used as the database for storing customer order details and managing records efficiently.

The backend files help establish communication between frontend pages and the database. These files manage customer orders, retrieve order information, and perform database operations such as storing and deleting records.

The major backend files used in the project are:

- "server.js"
- "order.js" (Models)
- "order.js" (Routes)

These files collectively ensure proper working of the Cafe Simulator application.

5.10.1 server.js

The "server.js" file is the main backend server file of the Cafe Simulator project. It is responsible for creating the server, connecting MongoDB database, handling middleware, and managing API routes.

The backend server is developed using Node.js and Express.js framework. Express middleware such as CORS and JSON parsing is used to process frontend requests efficiently.

The server file establishes communication between frontend pages and MongoDB database. It receives customer order data from frontend forms and stores the information securely in the database.

The "server.js" file also defines server port configuration and starts the application server successfully.

Features of server.js

- Express Server Creation
- MongoDB Connection
- Middleware Configuration
- API Route Integration
- JSON Data Handling
- Backend Server Management
- Frontend and Database Communication

5.10.1 server.js code:

```
const express = require("express");
const cors = require("cors");
const path = require("path");
require("./config/db");

const app = express();

app.use(cors());
app.use(express.json());


app.use(express.static(path.join(_dirname, "../cafe simulator")));
// Routes
const authRoutes = require("./routes/auth");
const orderRoutes = require("./routes/order");

app.use("/api", authRoutes);
app.use("/api", orderRoutes);

app.listen(5000, () => {
   console.log("Server running on port 5000");
});
```

5.10.2 order.js (models)

The "order.js" model file is used for defining the MongoDB schema structure of customer orders. It specifies how order data is stored in the MongoDB database.

The schema contains important customer details such as:

- Customer Name

- Phone Number

- Address

- Ordered Items

- Payment Method

- Total Amount

- Date and Time

The model file is created using Mongoose, which provides an easy way to interact with MongoDB collections and documents.

Features of order.js (Models)

- MongoDB Schema Definition
- Customer Data Structure
- Order Information Storage
- Database Validation
- Data Modelling using Mongoose

5.10.2 order.js (models) code:

```
const mongoose = require("mongoose");

const orderSchema = new

mongoose.Schema({ items: Array, total:

Number,

  name: String,

  phone: String,

  address: String,

  paymentMethod: String,

  date: {

    type: Date,

    default: Date.now

  }

});


module.exports = mongoose.model("Order", orderSchema);
```

5.10.3 order.js (routes)

The "order.js" routes file is responsible for handling API requests related to customer orders. It manages backend operations such as storing new orders, retrieving order details, and deleting records from the database.

The routes file receives requests from frontend pages using Fetch API and processes them through Express.js routing methods such as GET, POST, and DELETE.

This file acts as the connection layer between frontend user actions and MongoDB database operations.

Features of order.js (Routes)

- GET API Requests
- POST API Requests
- DELETE API Requests
- Order Data Retrieval
- Order Data Storage
- Database Communication
- API Route Handling

5.10.3 order.js (routes) code:

```
const express = require("express");
const router = express.Router();

const Order = require("../models/order");

// POST API to save order
router.post("/order", async (req, res) =>
{ try {
   const newOrder = new Order(req.body);
   await newOrder.save();

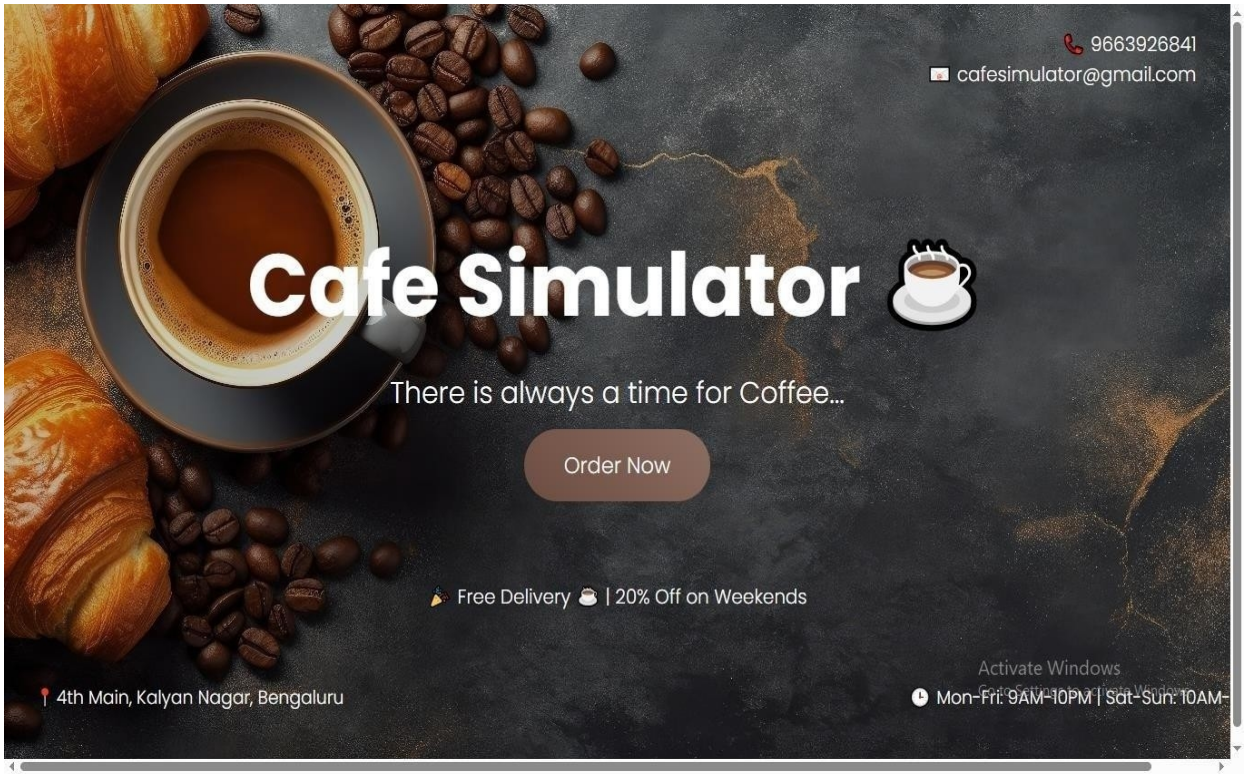
   res.status(200).json({ message: "Order saved successfully" });
  } catch (error)
   { console.error(error)
   ;
   res.status(500).json({ message: "Error saving order" });
  }
});

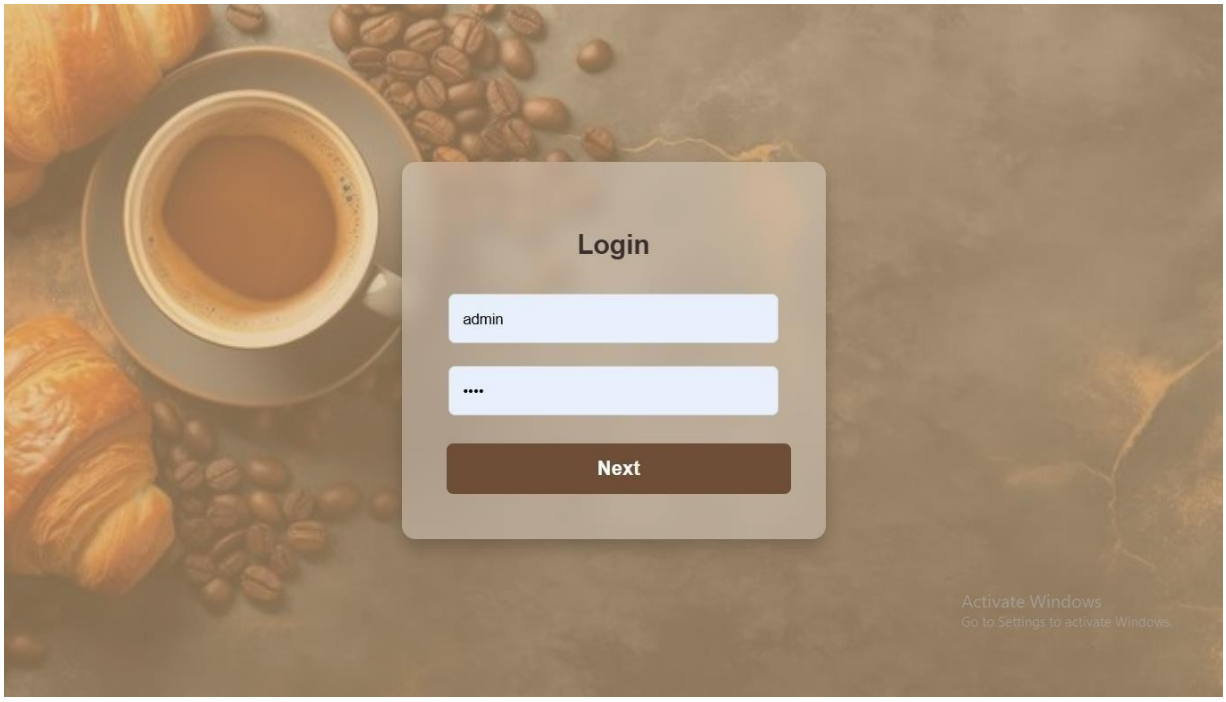
// GET API to fetch all orders
router.get("/orders", async (req, res) =>
  { try {
```

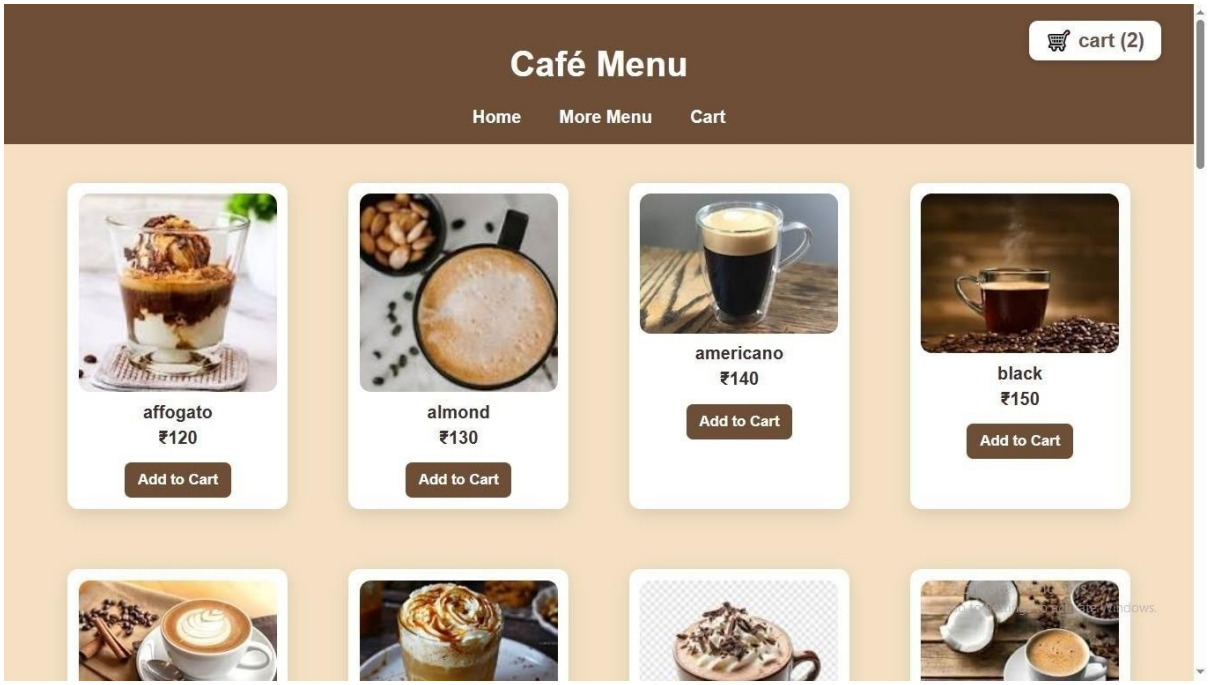
```
    const orders = await Order.find();
    res.status(200).json(orders);
  } catch (error)
    { console.error(error)
    ;
    res.status(500).json({ message: "Error fetching orders" });
  }
});

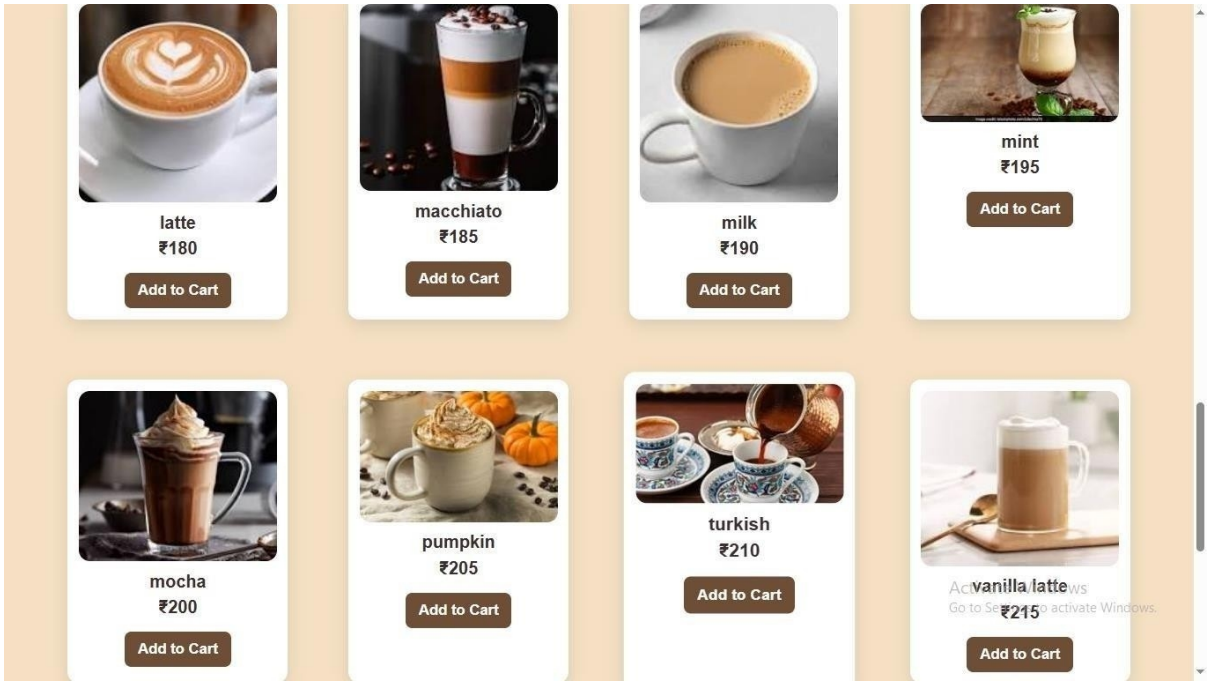
module.exports = router;

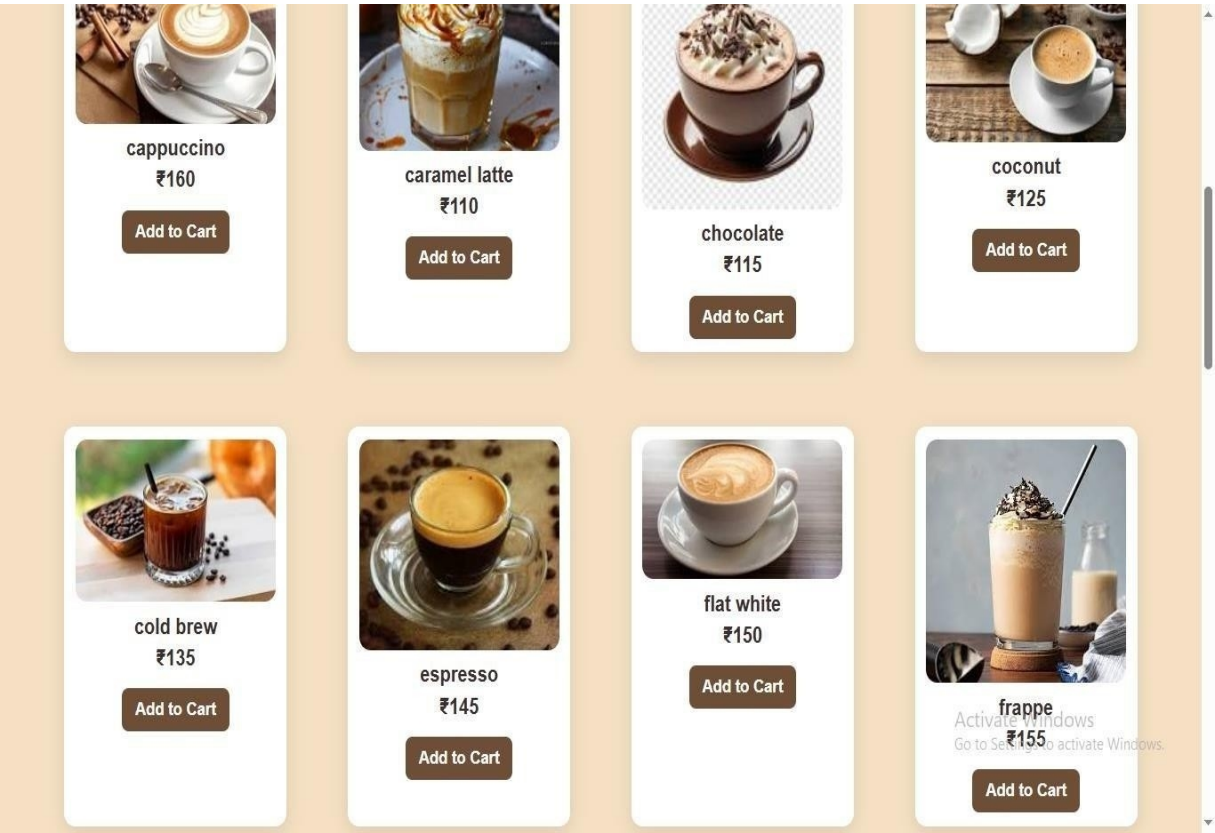

// DELETE API to remove order
router.delete("/order/:id", async (req, res) =>
{ try {
    await Order.findByIdAndDelete(req.params.id);
    res.status(200).json({ message: "Order deleted successfully" });
  } catch (error)
    { console.error(error)
    ;
    res.status(500).json({ message: "Error deleting order" });
  }
});
```

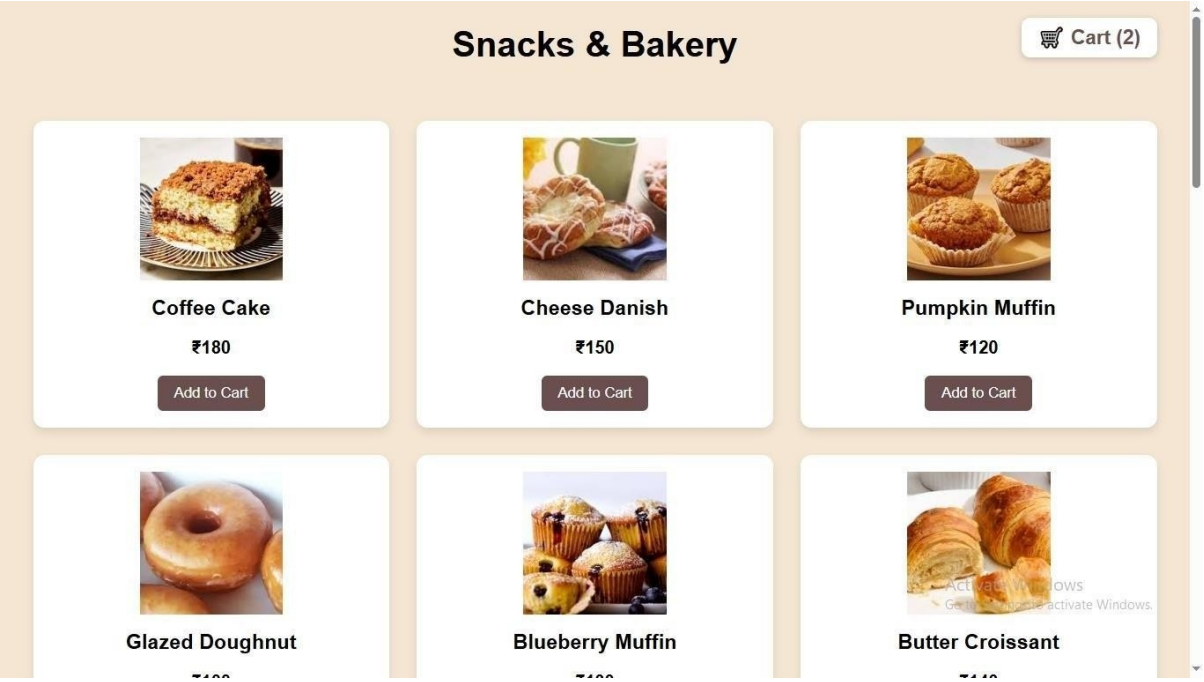

9663926841
cafesimulator@gmail.com
Cafe Simulator
There is always a time for Coffee...
Order Now
Free Delivery | 20% Off on Weekends
4th Main, Kalyan Nagar, Bengaluru
Mon-Fri: 9AM-10PM | Sat-Sun: 10AM-
Activate Windows

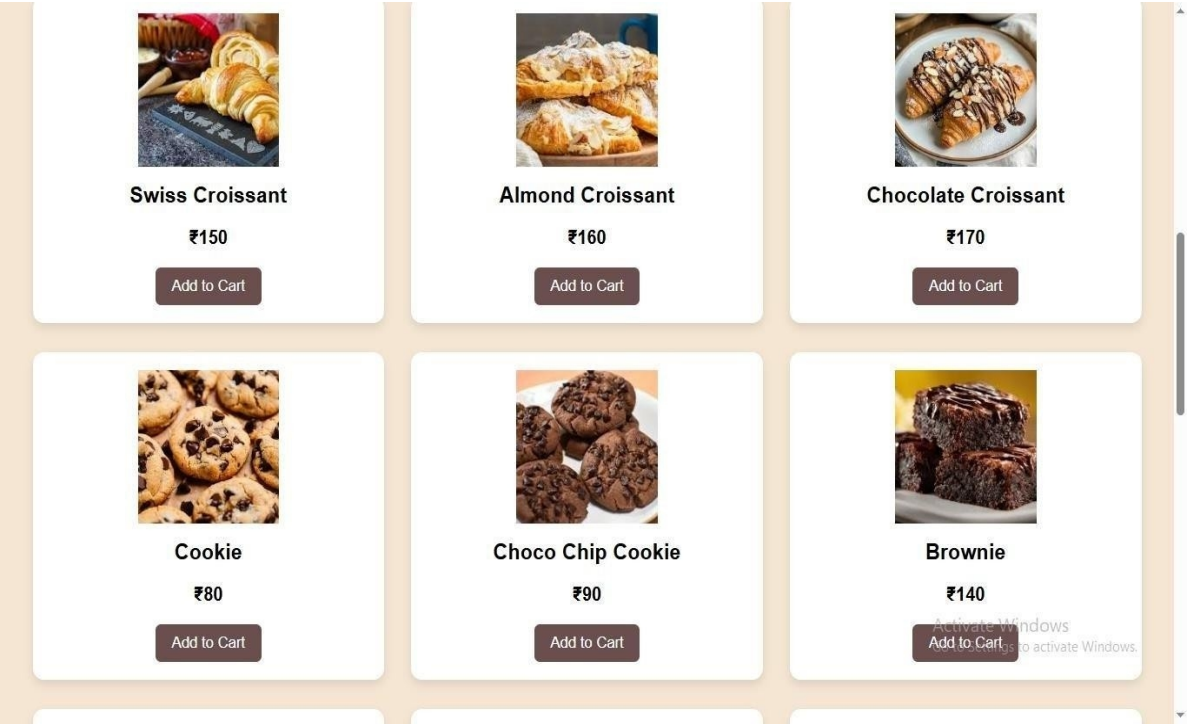

Login
admin
Next
Activate Windows
Go to Settings to activate Windows.

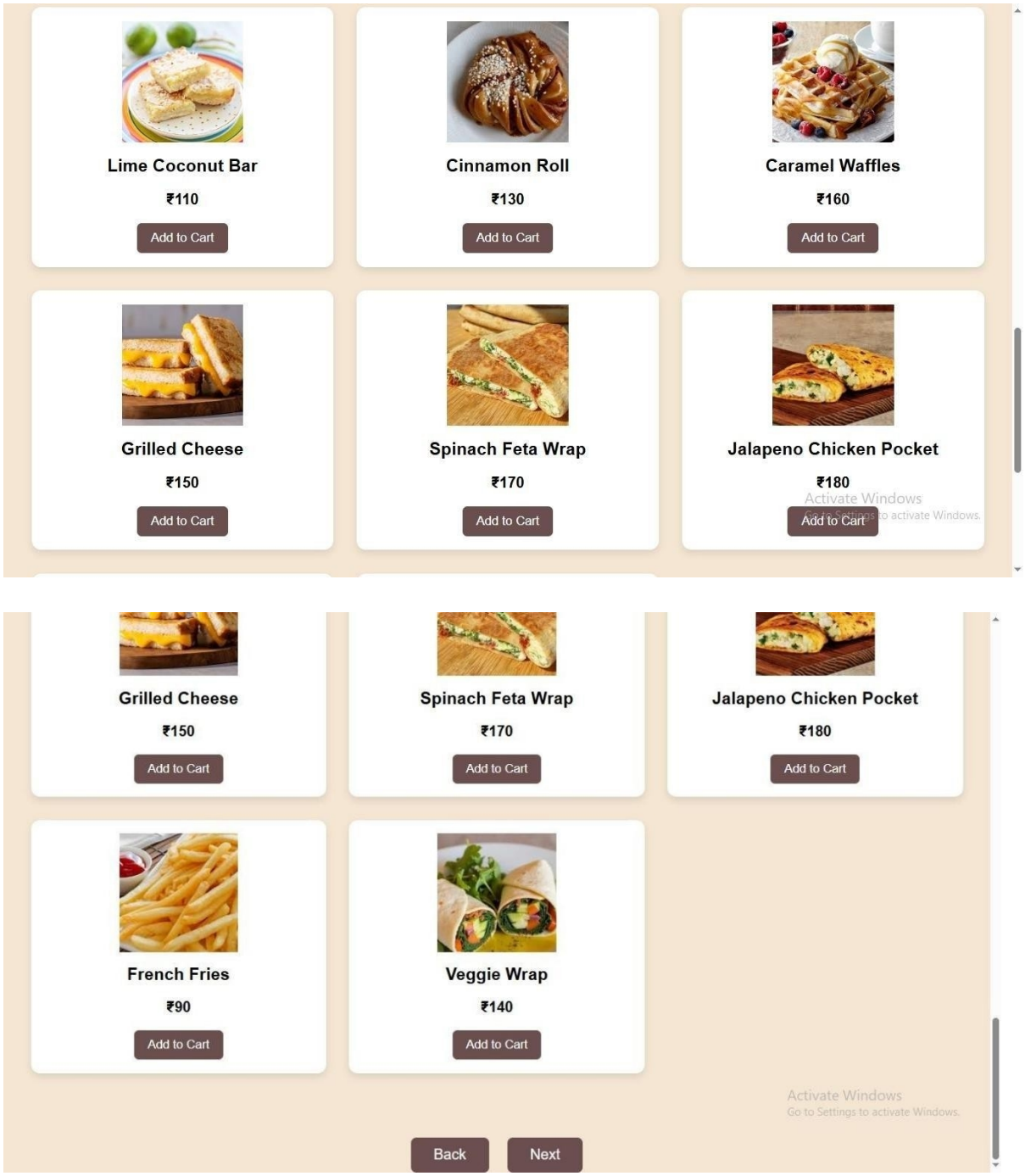

cart (2)
Café Menu
Home
More Menu
Cart
affogato
₹120
Add to Cart
almond
₹130
Add to Cart
americano
₹140
Add to Cart
black
₹150
Add to Cart

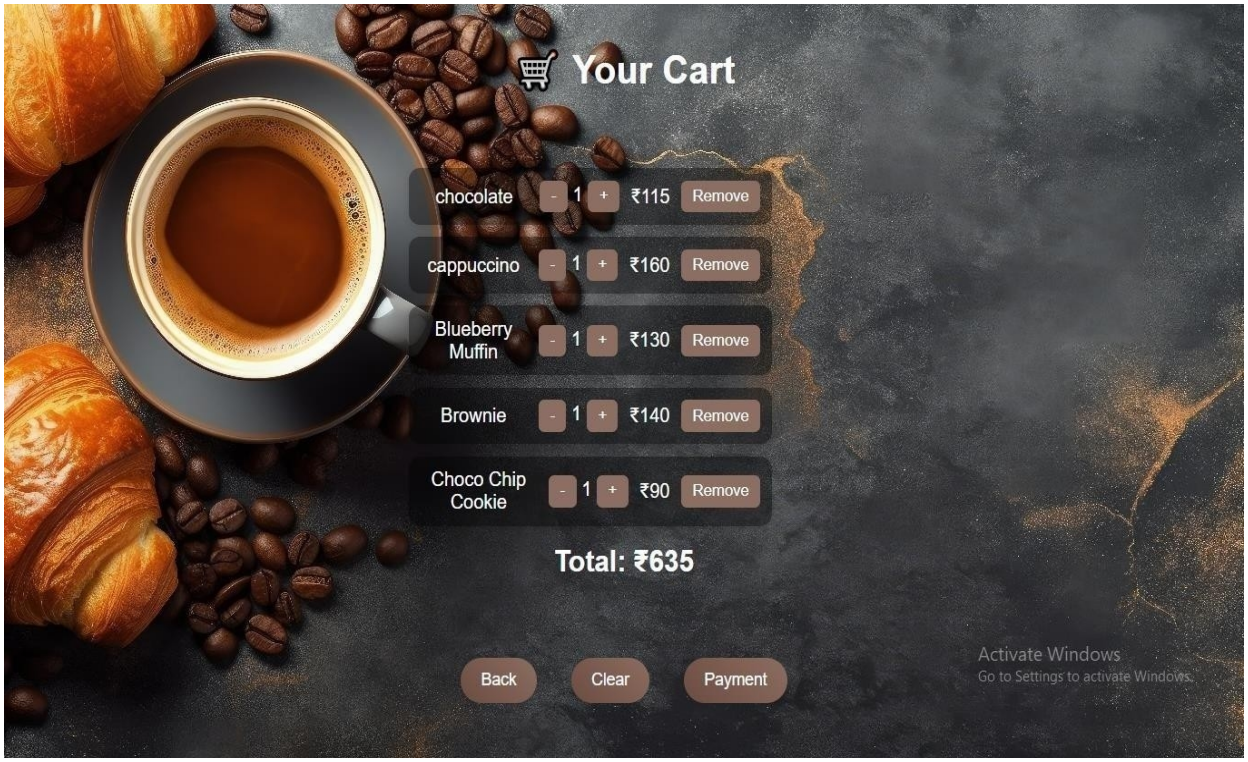

latte
₹180
Add to Cart
macchiato
₹185
Add to Cart
milk
₹190
Add to Cart
mint
₹195
Add to Cart
mocha
₹200
Add to Cart
pumpkin
₹205
Add to Cart
turkish
₹210
Add to Cart
vanilla latte
₹215
Add to Cart

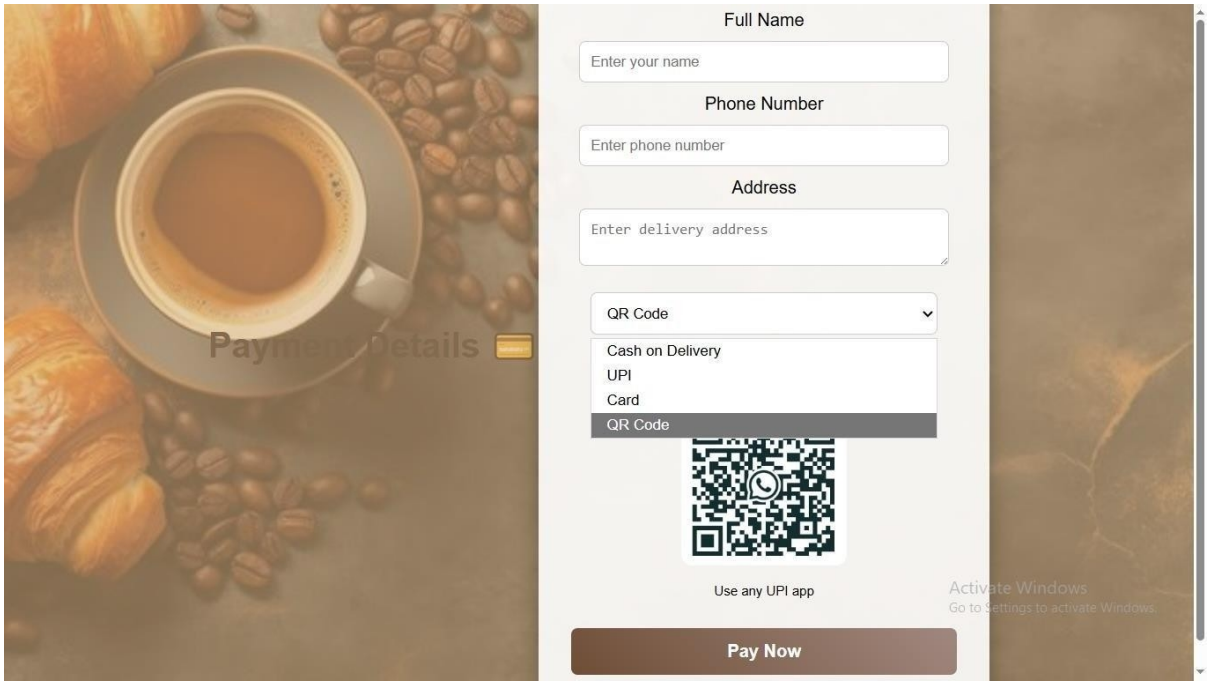

cappuccino
₹160
Add to Cart
caramel latte
₹110
Add to Cart
chocolate
₹115
Add to Cart
coconut
₹125
Add to Cart
cold brew
₹135
Add to Cart
espresso
₹145
Add to Cart
flat white
₹150
Add to Cart
frappe
₹155
Add to Cart

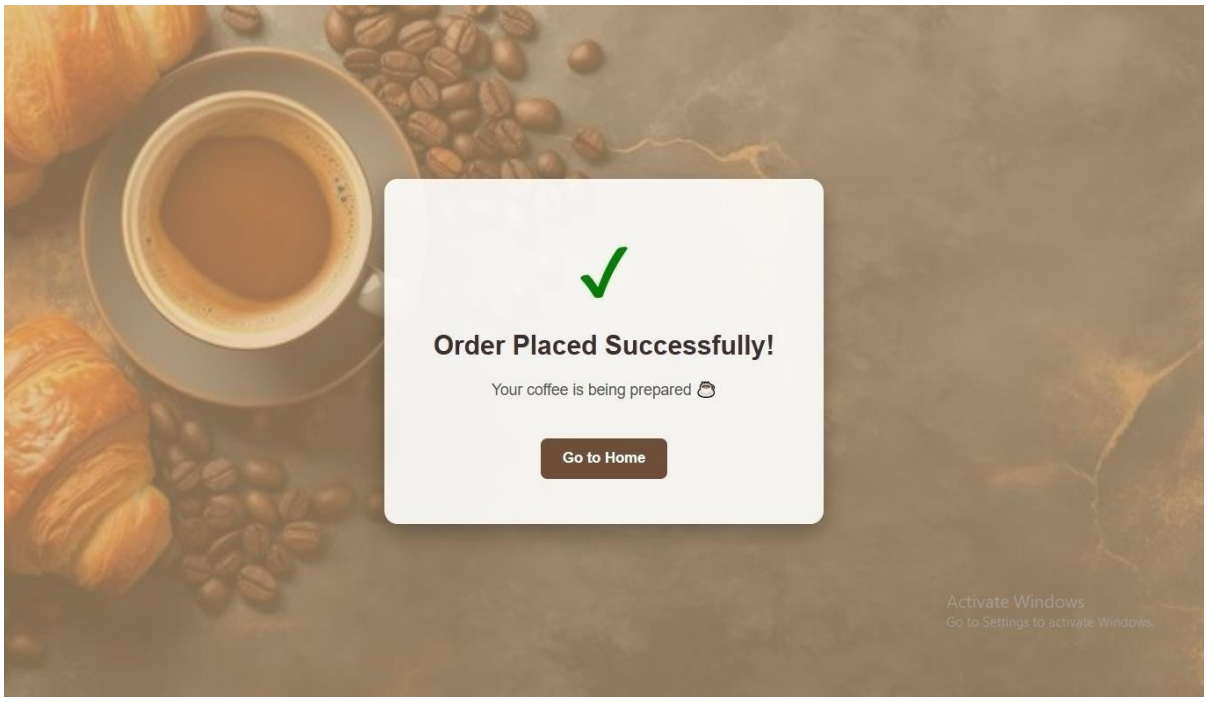

Snacks & Bakery
Cart (2)
Coffee Cake
₹180
Add to Cart
Cheese Danish
₹150
Add to Cart
Pumpkin Muffin
₹120
Add to Cart
Glazed Doughnut
Blueberry Muffin
Butter Croissant

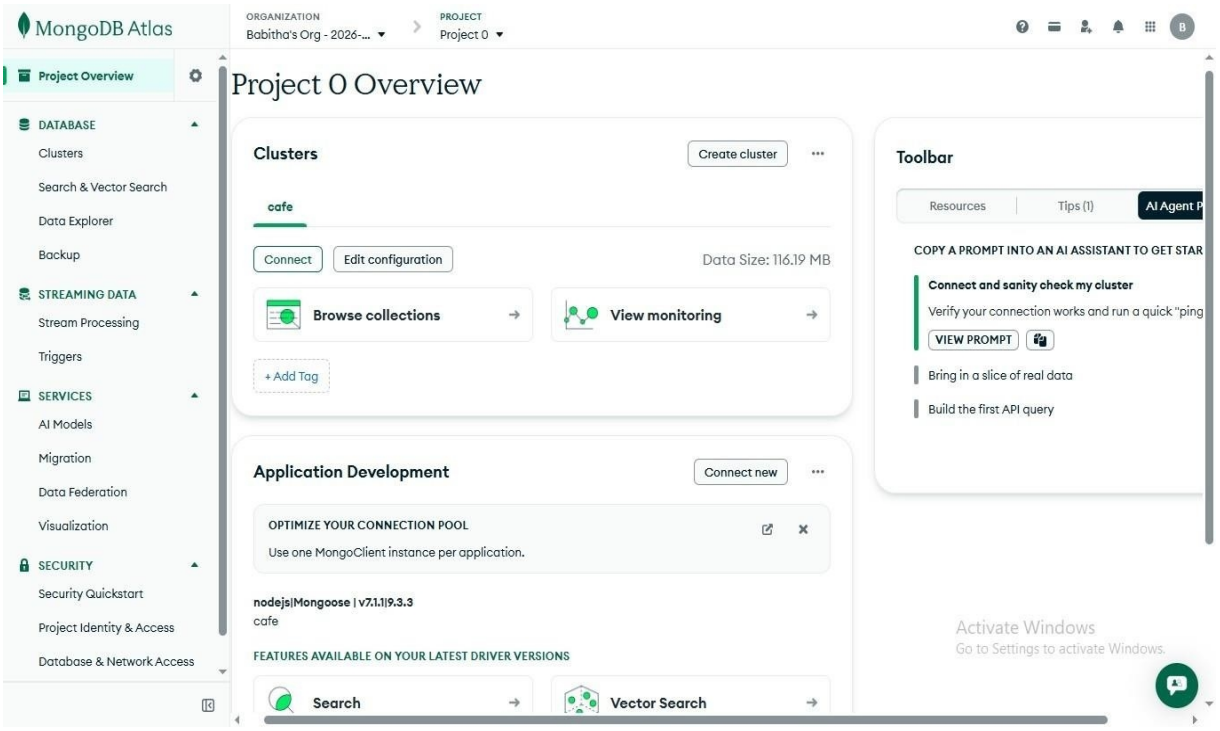

Swiss Croissant
₹150
Add to Cart
Almond Croissant
₹160
Add to Cart
Chocolate Croissant
₹170
Add to Cart
Cookie
₹80
Add to Cart
Choco Chip Cookie
₹90
Add to Cart
Brownie
₹140
Add to Cart

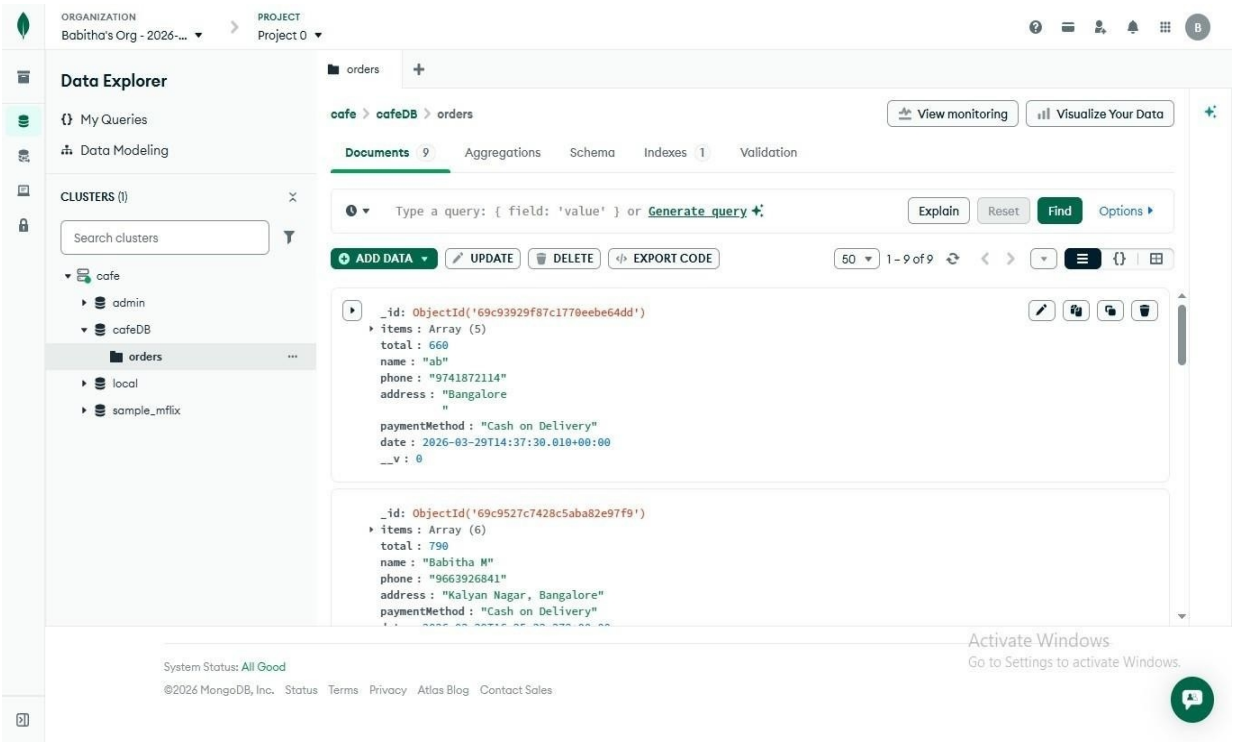

Lime Coconut Bar
₹110
Add to Cart
Cinnamon Roll
₹130
Add to Cart
Caramel Waffles
₹160
Add to Cart
Grilled Cheese
₹150
Add to Cart
Spinach Feta Wrap
₹170
Add to Cart
Jalapeno Chicken Pocket
₹180
Add to Cart
Activate Windows
Grilled Cheese
₹150
Add to Cart
Spinach Feta Wrap
₹170
Add to Cart
Jalapeno Chicken Pocket
₹180
Add to Cart
French Fries
₹90
Add to Cart
Veggie Wrap
₹140
Add to Cart
Activate Windows
Go to Settings to activate Windows.
Back
Next

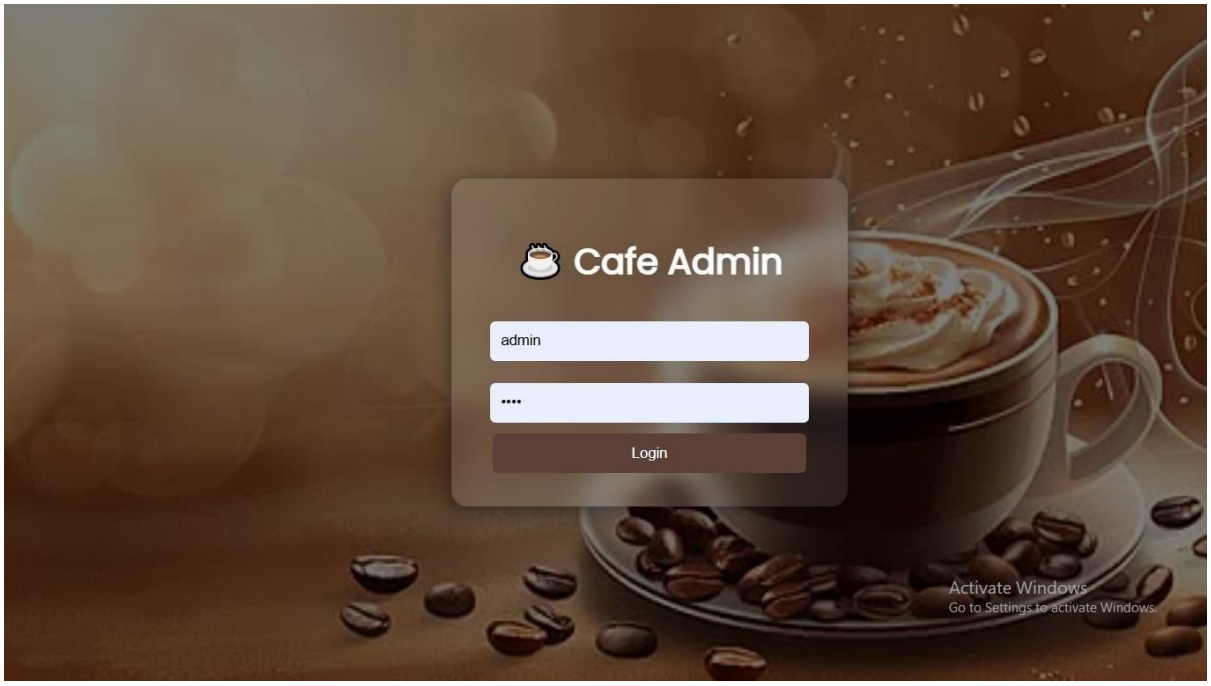

Your Cart
chocolate - 1 + ₹115 Remove
cappuccino - 1 + ₹160 Remove
Blueberry Muffin - 1 + ₹130 Remove
Brownie - 1 + ₹140 Remove
Choco Chip Cookie - 1 + ₹90 Remove
Total: ₹635
Back
Clear
Payment
Activate Windows
Go to Settings to activate Windows.

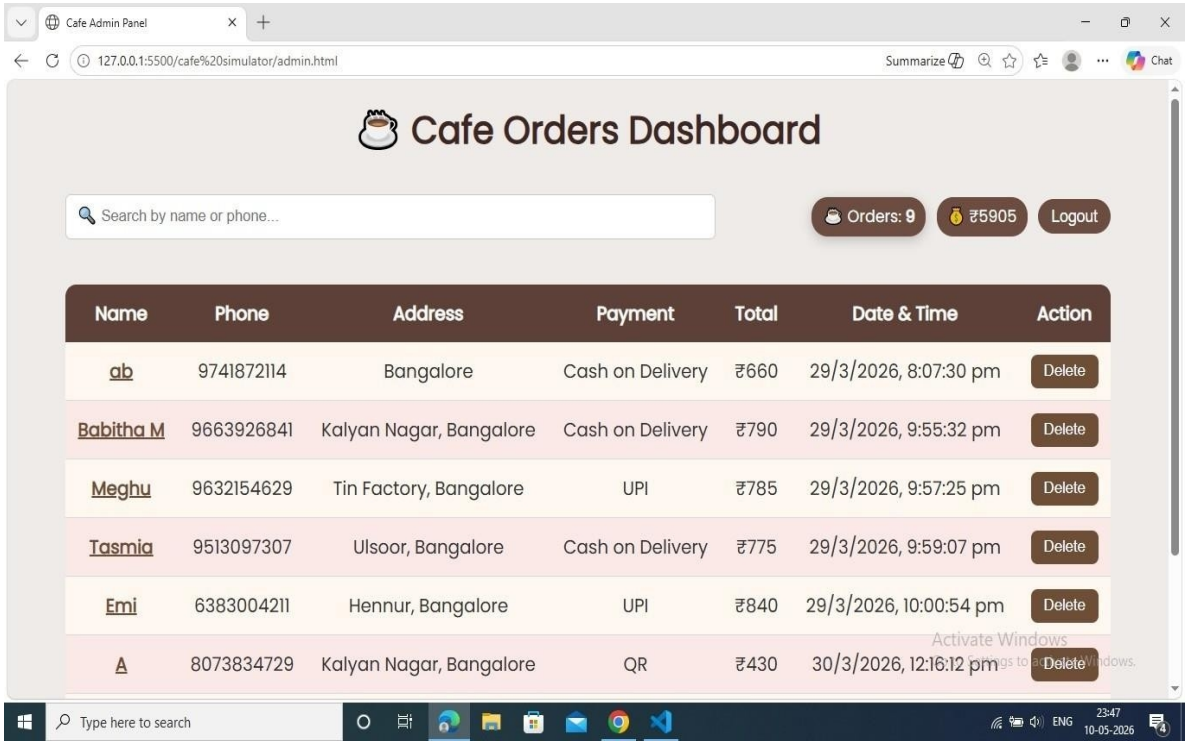

Payment Details
Full Name
Enter your name
Phone Number
Enter phone number
Address
Enter delivery address
QR Code
Cash on Delivery
UPI
Card
QR Code
Use any UPI app
Pay Now
Activate Windows
Go to Settings to activate Windows.

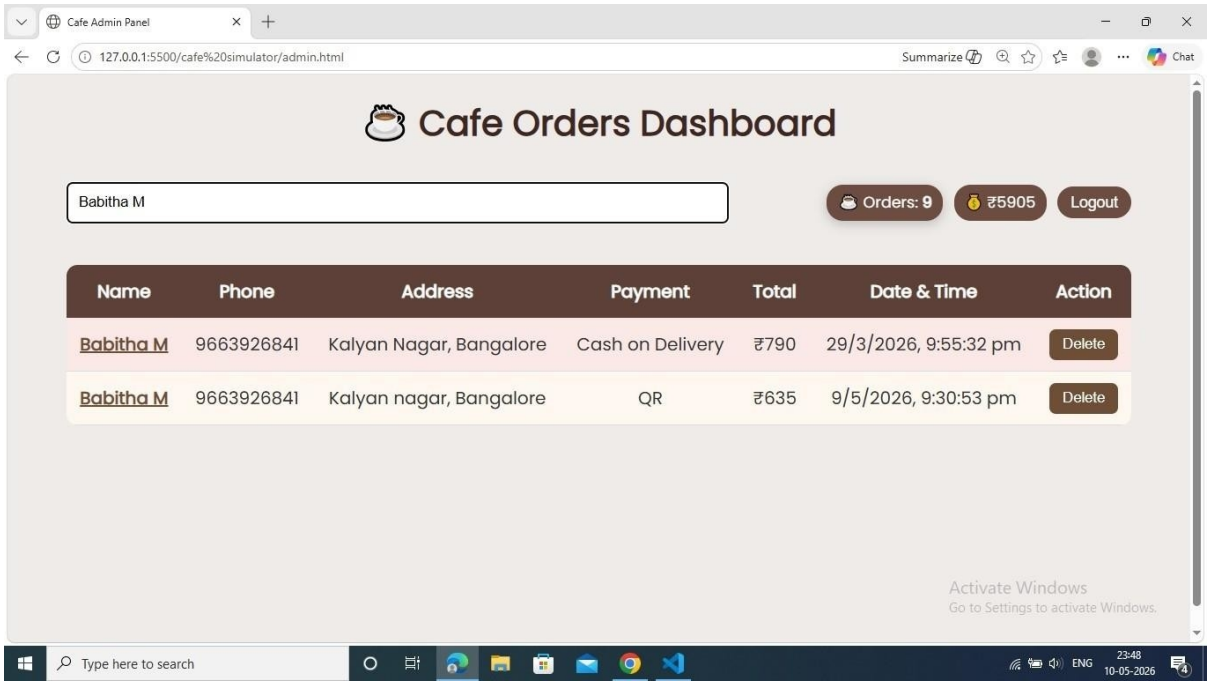

Order Placed Successfully!
Your coffee is being prepared
Go to Home
Activate Windows
Go to Settings to activate Windows.

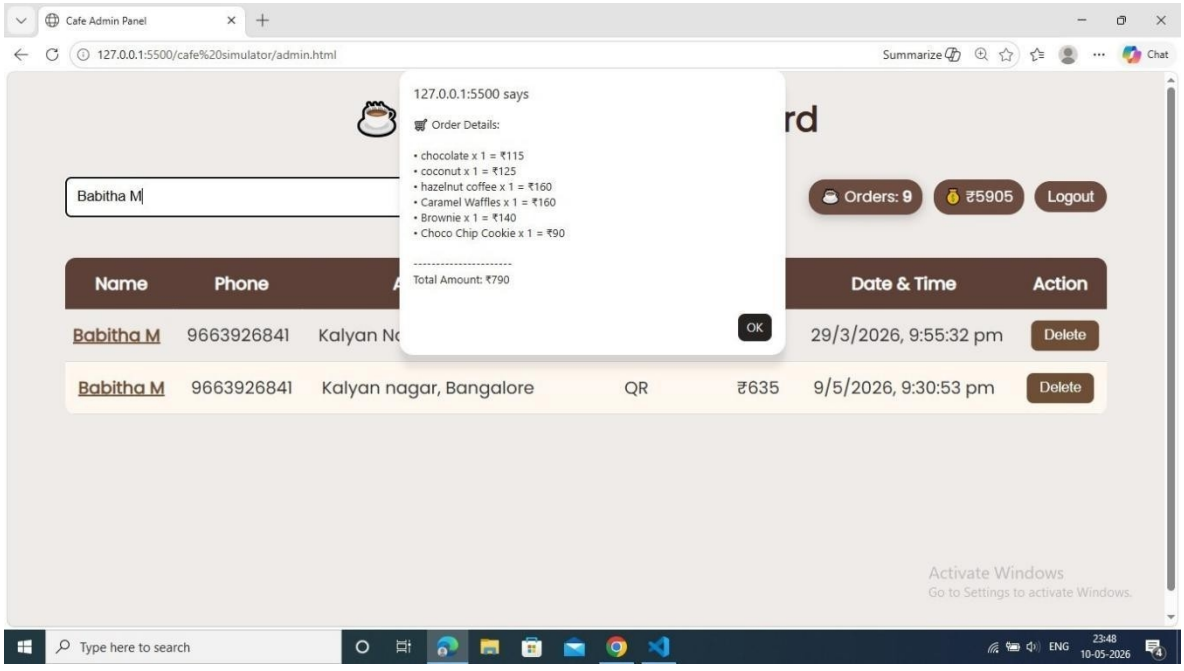

MongoDB Atlas
ORGANIZATION
Babitha's Org - 2026-...
PROJECT
Project 0
Project Overview
DATABASE
Clusters
Search & Vector Search
Data Explorer
Backup
STREAMING DATA
Stream Processing
Triggers
SERVICES
AI Models
Migration
Data Federation
Visualization
SECURITY
Security Quickstart
Project Identity & Access
Database & Network Access
Project 0 Overview
Clusters
Create cluster
cafe
Connect
Edit configuration
Data Size: 116.19 MB
Browse collections
View monitoring
+ Add Tag
Application Development
Connect new
OPTIMIZE YOUR CONNECTION POOL
Use one MongoClient instance per application.
nodejs|Mongoose | v7.1.1|9.3.3
cafe
FEATURES AVAILABLE ON YOUR LATEST DRIVER VERSIONS
Search
Vector Search
Toolbar
Resources
Tips (1)
AI Agent
COPY A PROMPT INTO AN AI ASSISTANT TO GET STAR
Connect and sanity check my cluster
Verify your connection works and run a quick "ping
VIEW PROMPT
Bring in a slice of real data
Build the first API query
Activate Windows
Go to Settings to activate Windows.


ORGANIZATION
Babitha's Org - 2026-...
PROJECT
Project 0
Data Explorer
My Queries
Data Modeling
CLUSTERS (1)
Search clusters
cafe
admin
cafeDB
orders
local
sample_mflix
orders
cafe > cafeDB > orders
View monitoring
Visualize Your Data
Documents 9
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
UPDATE
DELETE
EXPORT CODE
50
1 – 9 of 9
_id: ObjectId('69c93929f87c1770eebe64dd')
items : Array (5)
total : 660
name : "ab"
phone : "9741872114"
address : "Bangalore
"
paymentMethod : "Cash on Delivery"
date : 2026-03-29T14:37:30.010+00:00
__v : 0
_id: ObjectId('69c9527c7428c5aba82e97f9')
items : Array (6)
total : 790
name : "Babitha M"
phone : "9663926841"
address : "Kalyan Nagar, Bangalore"
paymentMethod : "Cash on Delivery"
System Status: All Good
©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales
Activate Windows
Go to Settings to activate Windows.


Cafe Admin
admin
Login
Activate Windows
Go to Settings to activate Windows.


Cafe Admin Panel
127.0.0.1:5500/cafe%20simulator/admin.html
Summarize
Chat
Cafe Orders Dashboard
Search by name or phone...
Orders: 9
₹5905
Logout
Name
Phone
Address
Payment
Total
Date & Time
Action
ab
9741872114
Bangalore
Cash on Delivery
₹660
29/3/2026, 8:07:30 pm
Delete
Babitha M
9663926841
Kalyan Nagar, Bangalore
Cash on Delivery
₹790
29/3/2026, 9:55:32 pm
Delete
Meghu
9632154629
Tin Factory, Bangalore
UPI
₹785
29/3/2026, 9:57:25 pm
Delete
Tasmia
9513097307
Ulsoor, Bangalore
Cash on Delivery
₹775
29/3/2026, 9:59:07 pm
Delete
Emi
6383004211
Hennur, Bangalore
UPI
₹840
29/3/2026, 10:00:54 pm
Delete
A
8073834729
Kalyan Nagar, Bangalore
QR
₹430
30/3/2026, 12:16:12 pm
Delete
Activate Windows
Type here to search
ENG
23:47
10-05-2026


Cafe Admin Panel
127.0.0.1:5500/cafe%20simulator/admin.html
Summarize
Chat
Cafe Orders Dashboard
Babitha M
Orders: 9
₹5905
Logout
Name
Phone
Address
Payment
Total
Date & Time
Action
Babitha M
9663926841
Kalyan Nagar, Bangalore
Cash on Delivery
₹790
29/3/2026, 9:55:32 pm
Delete
Babitha M
9663926841
Kalyan nagar, Bangalore
QR
₹635
9/5/2026, 9:30:53 pm
Delete
Activate Windows
Go to Settings to activate Windows.
Type here to search
ENG
23:48
10-05-2026


Cafe Admin Panel
127.0.0.1:5500/cafe%20simulator/admin.html
Summarize
Chat
127.0.0.1:5500 says
Order Details:
• chocolate x 1 = ₹115
• coconut x 1 = ₹125
• hazelnut coffee x 1 = ₹160
• Caramel Waffles x 1 = ₹160
• Brownie x 1 = ₹140
• Choco Chip Cookie x 1 = ₹90
Total Amount: ₹790
OK
Babitha M
Orders: 9
₹5905
Logout
Name
Phone
Date & Time
Action
Babitha M
9663926841
29/3/2026, 9:55:32 pm
Delete
Babitha M
9663926841
Kalyan nagar, Bangalore
QR
₹635
9/5/2026, 9:30:53 pm
Delete
Activate Windows
Go to Settings to activate Windows.
Type here to search
ENG
23:48
10-05-2026

CHAPTER-06

TESTING

6.1 TEST CASES

Test cases are used to verify whether the Cafe Simulator system performs according to the expected requirements. Different modules of the application were tested individually to ensure proper working of frontend pages, backend connectivity, database operations, payment processing, and admin functionalities.

The testing process helped identify and remove errors from the system. Each functionality was tested using valid and invalid inputs to verify system reliability and performance.

The following test cases were performed during system testing:

- Login Page Testing
- Menu Page Testing
- Add to Cart Testing
- Cart Quantity Testing
- Remove Item Testing
- Payment Validation Testing
- MongoDB Database Testing
- Admin Dashboard Testing
- Search Functionality Testing
- Delete Functionality Testing
- Responsive Design Testing
- Notification Popup Testing

6.2 RESULTS

- Successful Login Validation
- Successful Cart Operations
- Successful Payment Processing
- Successful Database Storage
- Successful Order Retrieval
- Successful Admin Dashboard Display
- Successful Search Functionality
- Successful Delete Functionality
- Responsive UI Results
- Backend API Communication Results
- MongoDB Connection Results
- Overall System Performance Results

CHAPTER-07

RESULTS AND DISCUSSION

7.1 RESULTS

The Cafe Simulator project was successfully developed and implemented using HTML, CSS, JavaScript, Node.js, Express.js, and MongoDB technologies. The system performs all major functionalities efficiently and provides a smooth user experience for both customers and administrators.

The application successfully allows customers to browse menu items, add products to the cart, update quantities, and complete the payment process. Customer details such as name, phone number, address, payment method, ordered items, and total amount are stored successfully in the MongoDB database through backend API integration.

The admin dashboard functions properly and displays all customer order details clearly. The search functionality helps administrators search orders using customer names or phone numbers. The delete functionality also works successfully for removing unwanted order records from the database.

The frontend pages are responsive and display properly on different devices and screen sizes. Attractive UI design, hover effects, animations, and coffee-themed backgrounds improve the visual appearance of the project. The integration between frontend, backend, and database was tested successfully, and the system worked efficiently without major errors.

7.2 DISCUSSION

The Cafe Simulator project demonstrates the practical implementation of full-stack web development technologies for building an online cafe ordering and management system. The project successfully integrates frontend technologies with backend server operations and database management.

The use of HTML and CSS helped create an attractive and responsive user interface, while JavaScript provided dynamic functionalities such as cart management, notifications, and validations. Node.js and Express.js handled backend operations and API communication efficiently. MongoDB was used for secure storage and retrieval of customer order details.

The project improves cafe management by reducing manual work and simplifying customer order handling. The admin dashboard helps monitor customer orders efficiently and provides easy access to database records. Features such as search functionality, delete operation, and order item display improve system usability.

Although the project performs efficiently, certain advanced features such as online payment gateway integration, live order tracking, customer authentication, and inventory management can be added in future versions for further improvement. Overall, the Cafe Simulator project successfully achieves its objectives and provides an efficient digital solution for cafe order management.

CHAPTER-08

FUTURE ENHANCEMENT

FUTURE ENHANCEMENT

The Cafe Simulator project was successfully developed with all basic functionalities required for cafe order management. However, several advanced features can be added in future versions to improve the efficiency, security, and usability of the system.

One of the major future enhancements is the integration of an online payment gateway such as Google Pay, PhonePe, Paytm, or credit/debit card payment systems. This feature would allow customers to make secure online payments directly through the application.

User authentication and account management features can also be added in future versions. Customers can create personal accounts, log in securely, and view their previous order history. This would improve personalization and customer experience.

Another important enhancement is live order tracking. Customers can track the status of their orders such as “Order Confirmed,” “Preparing,” “Out for Delivery,” and “Delivered.” This feature would improve transparency and customer satisfaction.

The system can also be enhanced by adding inventory management functionality. Admins can monitor available stock, update product availability, and receive notifications when stock levels are low.

Email and SMS notification features can also be integrated to send order confirmations and payment updates automatically to customers. This would improve communication between customers and cafe management.

In future versions, AI-based recommendation systems can also be implemented to suggest menu items based on customer preferences and order history. Mobile application support for Android and iOS platforms can further improve accessibility and user convenience.

Overall, these future enhancements can make the Cafe Simulator project more advanced, secure, scalable, and suitable for real-world commercial cafe management systems.

CONCLUSION

The **Cafe Simulator** project was successfully designed and developed as a web-based café ordering and management system using **HTML, CSS, JavaScript**, **Node.js, Express.js, and MongoDB technologies.** The project provides an efficient digital platform for customers to browse menu items, manage carts, make payments, and place orders easily.

The system successfully integrates frontend interfaces, backend server operations, and database management to provide smooth communication between users and administrators. The application simplifies cafe order handling by reducing manual work and improving order management efficiency.

The admin dashboard helps administrators monitor customer orders, search records, delete unnecessary entries, and view complete order details efficiently. Features such as responsive design, dynamic cart management, payment validation, and database connectivity improve the overall performance and usability of the system.

The project also demonstrates practical implementation of full-stack web development concepts including API integration, responsive UI design, local Storage management, and MongoDB database operations.

Although the current system performs efficiently, additional features such as online payment gateways, live order tracking, customer authentication, and inventory management can be implemented in future versions for further enhancement.

Overall, the **Cafe Simulator** project successfully achieves its objectives and provides a modern, user-friendly, and efficient solution for cafe order management.

REFRENCES

1. **HTML Documentation** – Used for designing the structure of web pages.
2. **CSS Documentation** – Used for styling and responsive web design.
3. **JavaScript Documentation** – Used for implementing dynamic functionalities and client-side scripting.
4. **Node.js Documentation** – Used for backend server development.
5. **Express.js Documentation** – Used for API routing and server-side operations.
6. **MongoDB Documentation** – Used for storing customer order data.
7. **Visual Studio Code** – Used for project development and code editing.
8. **Google Search Resources** – Used for collecting technical references and troubleshooting.
9. **GitHub Resources** – Used for learning project implementation and coding references.
10. **Web Development Tutorials and Online Learning Resources related to Full** - Stack Development and MongoDB Integration.